Tunisian Republic
Ministry of Higher Education and
Scientific Research

University of Sfax
National Engineering School of Sfax

Major: Computer Engineering

Internship Report

Presented in :

National Engineering School of Sfax
Department of Computer Engineering and Applied Mathematics

Subject :

Design and Implementation of a Multi-Agent AI Platform for Agricultural Decision Support

Reported by :

Mariem Kbaier

Host Organization : Talan Tunisie
Internship Period : July – August

Academic supervisor : Bachar Zouari
Company supervisor : Imen Ayari

Academic Year: 2025/2026

TALAN
Positive innovation

Acknowledgements

This project would not have been possible without the assistance and support of various individuals, to whom I would like to express my gratitude and dedicate this work.

I would like to thank **DIEU** the almighty for helping me begin and complete this work.

Mr. **Bachar Zouari**, my academic supervisor at the National School of Engineers of Sfax, has consistently offered invaluable guidance and support. His unwavering dedication, constructive criticism, and pertinent suggestions have been instrumental in ensuring the success of this work and in keeping it well organized throughout.

I am equally grateful to Mrs. **Imen Ayari**, my company supervisor at **Talan Tunisie**, for her availability, the guidance she provided, and the trust she placed in me to work on this project. I also extend my thanks to the broader team at Talan Tunisie for welcoming me and supporting my work during this internship.

I would like to thank my project teammates for their collaboration, commitment, and the group effort that made this work possible.

I warmly thank all my professors at the National School of Engineers of Sfax for their continuous support and the valuable training provided throughout my years of study.

All esteemed contributors, whether through direct or indirect means, who have contributed to this project, deserve my sincere appreciation.

Finally, I extend my particular gratitude to all the members of the Jury for attending the presentation of this work.

Contents

1 Project Context and State Of The Art **1**

1.1 Introduction 1

1.2 Host Organization 1

1.3 Key Concepts 2

1.3.1 Precision Agriculture and Digital Decision Support 2

1.3.2 Large Language Models: LLMs 2

1.3.3 Retrieval-Augmented Generation (RAG) and Grounding 4

1.3.4 Web Scraping and Automated Corpus Acquisition 5

1.3.5 AI-Assisted Frontend Generation 9

1.3.6 3D Geospatial Visualization 10

1.3.7 Multi-Agent Systems and Orchestration 11

1.3.8 Human-in-the-Loop Decision Support 12

1.3.9 Machine Learning for Crop Suitability 13

1.3.10 Remote Sensing and Vegetation Indices 14

1.3.11 AgriAdvisor's Data Sources 15

1.3.12 Deterministic Agronomic and Financial Modeling 16

1.4 State of Art 17

1.4.1 Enhancing Agricultural Productivity: A Machine Learning Approach to Crop Recommendations 17

1.4.2 BreizhCrops: A Time Series Dataset for Crop Type Mapping 17

1.4.3 Reducing Hallucination in Structured Outputs via RAG 18

1.4.4 Discussion 18

1.5 Conclusion 19

2 Contribution: AgriAdvisor – A Multi-Agent Platform for Agricultural Decision Support 20

2.1 Introduction . . . 20
2.2 Objective of the Contribution . . . 20
2.3 Methodology . . . 21
2.3.1 Data Collection and Knowledge Base Construction . . . 21
2.3.2 Agent Design and Grounding Strategy . . . 22
2.3.3 Alignment with the EU AI Act . . . 23
2.3.4 Model Selection . . . 24
2.4 Platform Architecture and Functional Requirements . . . 24
2.4.1 Architectural Overview . . . 24
2.4.2 Platform Functional Requirements . . . 25
2.4.3 Deterministic Agronomic and Financial Calculation Engine . . . 27
2.4.4 Agro-Security and LLM Safety Guardrails (AgroLLM-Sec) . . . 29
2.5 Use Case Diagram . . . 30
2.6 Conclusion . . . 31

3 Evaluation and Implementation of AgriAdvisor 36

3.1 Introduction . . . 36
3.2 Evaluation Methodology and Criteria . . . 36
3.3 Evaluation of the Regulation Agent . . . 37
3.3.1 Gold-Standard Question Set . . . 37
3.3.2 Retrieval Configuration Comparison . . . 38
3.3.3 Multi-Rank Retrieval Precision (Hit@k) . . . 38
3.3.4 End-to-End Latency Breakdown . . . 39
3.3.5 LLM Judge Validation . . . 40
3.3.6 Statistical Significance and Confidence Intervals . . . 40
3.3.7 Groundedness (Faithfulness) Metric and No-RAG Baseline Comparison 41
3.3.8 Tool-Selection Accuracy . . . 42
3.3.9 Evaluation of the AgroLLM-Sec Guardrails . . . 43
3.4 Evaluation of the Agriculture Agent . . . 43
3.4.1 Crop Suitability Scorer Evaluation . . . 43
3.4.2 DL Crop-Cover Classifier Evaluation . . . 43

3.4.3 Report Groundedness and Numeric Audit Evaluation 44
3.5 Evaluation of the Business and Trading Agents 46
3.5.1 Financial Calculation Correctness 46
3.5.2 Scoring Sensitivity Analysis 46
3.6 Final Configuration Selection 46
3.7 Implementation 48
3.7.1 Hardware Environment 48
3.7.2 Software Environment and Tools 48
3.7.3 Chunking, Embedding, and Indexing Pipeline 49
3.8 Graphical User Interfaces 52
3.8.1 Parcel Analysis Page 53
3.8.2 Regulation Chat Interface 54
3.8.3 Business Scenarios Page 54
3.8.4 Trading Dashboard 55
3.9 Deployment 56
3.10 Conclusion 56

General Conclusion **57**

Bibliography **57**

List of Figures

1.1 Hybrid search combining dense and sparse retrieval 5
1.2 Web scraping and automated corpus acquisition pipeline 8
1.3 From 2D map data to interactive 3D geospatial visualization using Three.js and Mapbox GL JS 10
1.4 Dynamic agent handoff and control-shift routing 12
1.5 Remote sensing and vegetation indices used by AgriAdvisor 14

2.1 AgriAdvisor Platform Architecture 24
2.2 AgroLLM-Sec safety layer: input filtering and output validation 30
2.3 AgriAdvisor Use Case Diagram 31

3.1 Regulation agent RAG evaluation pipeline and key findings 42
3.2 Numeric audit evaluation: error-injection procedure, results, and semantic-mismatch limitation 45
3.3 Chunking, embedding, and indexing pipeline for the FranceAgriMer corpus 51
3.4 Parcel Analysis Page: Map Selection and Crop Recommendations 53
3.5 Regulation Agent Chat Interface 54
3.6 Business Scenarios Comparison Page 54
3.7 Trading Dashboard: Commodity Tickers and Commercialization Strategy . 55

List of Tables

1.1 Summary of Reviewed Work Related to AgriAdvisor's Core Techniques . . 18

2.1 AgriAdvisor's Microservice Agents 25
2.2 Mid-season crop coefficients (K_c) used by the irrigation engine 27
2.3 Unit nitrogen requirement (b) by crop 28
2.4 Maximum admissible nitrogen dose by crop (Directive Nitrates / ZVN) . . 28
2.5 Weighting of the crop-suitability scorer's soil and climate criteria 29
2.6 Textual description of the "Ask Regulatory Question" use case 32
2.7 Textual description of the "Analyze Parcel and Recommend Crops" use case 33
2.8 Textual description of the "Generate Business Scenarios" use case 34
2.9 Textual description of the "Evaluate Commercialization Strategy" use case 35

3.1 Regulation Agent – Retrieval Configuration Comparison 38
3.2 Regulation Agent – Multi-Rank Retrieval Precision (Hit@k) 39
3.3 Regulation Agent – End-to-End Latency Breakdown 39
3.4 LLM Judge Validation – Human Expert vs. `open-mistral-7b` (N=6) . . . 40
3.5 Bootstrap 95% Confidence Intervals – Hit@5 41
3.6 Regulation Agent – Faithfulness by Configuration (RAG vs. No-RAG) . . 41
3.7 AgroLLM-Sec Safety Benchmark Results (N=50) 43
3.8 DL Crop-Cover Classifier – Fine-Tuning Results 44
3.9 Numeric Audit – Precision and Recall 44

Chapter 1

Project Context and State Of The Art

1.1 Introduction

In this chapter, we explore the foundational concepts essential for understanding AgriAdvisor, a multi-agent, AI-native decision-support platform for French farmers and agricultural buyers. We begin with a brief overview of precision agriculture and digital decision support, before examining Large Language Models (LLMs) and the Mistral models used throughout the platform. We then detail Retrieval-Augmented Generation (RAG) and the hybrid dense/sparse retrieval techniques used to ground LLM answers in official regulatory and agronomic sources, followed by the multi-agent orchestration pattern that structures the platform into specialized services. We next cover the machine learning and remote-sensing techniques used for crop suitability scoring and satellite-based crop classification, and close with the deterministic agronomic and financial modeling techniques used where a wrong number would carry real consequences. Finally, we review recent research that aligns with each of these areas in the State of the Art section, before positioning our contribution relative to existing work.

1.2 Host Organization

Talan [1] is an international consulting and technology group founded in 2002 and headquartered in Paris, France. The group specializes in digital transformation, data intelligence and innovation consulting, and operates across five continents in eighteen countries, employing more than 7,200 people worldwide. Talan places innovation at the center of its strategy through a dedicated Research and Innovation Center, active in emerging technology fields such as artificial intelligence, data intelligence and blockchain.

Talan Tunisia [2], created in 2008, is the group's nearshore development center, gathering more than 500 engineers from leading Tunisian and European engineering schools, working

on projects for major European clients. Its activity spans digital transformation, web and mobile development, systems integration, software testing, and application support. Talan Tunisia also hosts an Innovation Factory, a research and development structure through which it organizes technology bootcamps – including the Summer Camp program in which the present project was carried out – bringing together students, mentors and academic partners around applied projects in artificial intelligence and other disruptive technologies.

1.3 Key Concepts

This section presents the essential concepts that support the project: precision agriculture, Large Language Models, Retrieval-Augmented Generation, multi-agent systems, machine learning for agronomic decision support, remote sensing, and deterministic decision modeling. These foundations provide the technical context needed to understand AgriAdvisor's goals and architecture, detailed in the next chapter.

1.3.1 Precision Agriculture and Digital Decision Support

Precision agriculture [3] refers to the use of data, technology, and analytics to manage farming decisions — crop selection, fertilization, irrigation, and risk management — at a finer granularity than traditional, experience-based practice. In France specifically, farmers must additionally navigate a dense and frequently updated body of regulation (the Code Rural, the Politique Agricole Commune, CERFA administrative forms, and national and regional subsidy programs), alongside agronomic and financial decisions that carry real consequences if made on outdated or incomplete information. AgriAdvisor is designed as a digital decision-support platform addressing these needs across six domains: regulatory compliance, crop and geographic suitability, business planning, monitoring and alerts, agricultural waste valorization, and a community marketplace. The rest of this section introduces the technical building blocks — LLMs, RAG, multi-agent orchestration, machine learning, remote sensing, and deterministic modeling — that these domains are built on.

1.3.2 Large Language Models: LLMs

This part offers a comprehensive overview of LLMs, their architecture, and the specific models used throughout AgriAdvisor.

1.3.2.1 Definition

LLMs [4] are neural network-based AI systems that fall under the broader category of generative AI. They are trained on vast amounts of textual data, making them capable of understanding and generating human language to perform a wide range of tasks. The

"large" in their name refers both to the enormous datasets they train on and to their massive number of parameters (sometimes hundreds of billions) that define the model's behavior.

1.3.2.2 Architecture of Large Language Models

LLMs are based on the transformer architecture, which uses self-attention mechanisms to process input sequences in parallel. **Transformers** [5] capture relationships between words in a sequence, enabling them to understand and generate text effectively and to handle long-range dependencies efficiently.

Most modern LLMs, including the GPT, LLaMA, and Mistral families, are decoder-only architectures optimized for text generation. AgriAdvisor relies specifically on **Mistral AI**'s model family [6] — `mistral-small-latest` and `mistral-large-latest` for tool-calling and answer synthesis in the Regulation agent, `mistral-medium-latest` for the Agriculture agent's report generation and conversational assistant, and `mistral-embed` for dense embeddings — chosen for their strong performance on French-language text and their availability as an OpenAI-compatible API.

Key components of LLMs

- **Tokenization:** Breaking text into smaller units (tokens).
- **Embedding:** Converting tokens into numerical vector representations.
- **Attention Mechanisms:** Weighing the importance of different words in context.
- **Structured Output Generation:** Constraining an LLM's output to conform to a predefined schema (e.g., a Pydantic model), rather than free text, so downstream code can consume the result reliably [7].

1.3.2.3 Tool-Calling / Function-Calling Agents

A **tool-calling agent** [8] is an LLM augmented with a set of external tools (functions) it can choose to invoke based on the user's request, rather than answering solely from its own parametric knowledge. The model decides which tool to call, receives the tool's output, and incorporates it into its final answer. This pattern underlies the Regulation agent, whose LLM chooses between a hybrid-search tool over indexed regulation, a targeted subsidy-search tool, and a general web-search tool depending on the question — in contrast to a plain chatbot, which has no mechanism to consult external, up-to-date sources before answering.

1.3.3 Retrieval-Augmented Generation (RAG) and Grounding

1.3.3.1 Definition

Retrieval-Augmented Generation (RAG) [9] is a technique that combines a retrieval step — searching a document collection for passages relevant to a query — with a generation step, in which an LLM produces its answer conditioned on the retrieved passages rather than on its training data alone. RAG significantly reduces the tendency of LLMs to *hallucinate* — to state plausible-sounding but incorrect or fabricated information — since the model is grounded in verifiable, up-to-date source text [10]. This matters particularly in regulated or high-stakes domains, such as legal and agricultural-subsidy information, where an invented figure or an outdated rule can have real financial or legal consequences [10].

1.3.3.2 Dense Embeddings and Vector Databases

A **dense embedding** is a fixed-length numerical vector representation of a piece of text, positioned in a high-dimensional space such that semantically similar texts are close together. AgriAdvisor uses Mistral's `mistral-embed` model to produce 1024-dimensional embeddings for both indexed documents and incoming queries, stored and queried via **Qdrant** [11], a vector database purpose-built for high-performance similarity search, and via **Chroma** [12] for the Agriculture agent's document corpus.

1.3.3.3 Sparse Retrieval (BM25)

BM25 [13] is a classical, keyword-based (lexical) ranking function from information retrieval, scoring documents by term frequency and inverse document frequency rather than by semantic similarity. Unlike dense embeddings, BM25 is highly effective at matching exact terminology — article numbers, legal references, or precise agronomic terms — that a purely semantic search might overlook. AgriAdvisor's Regulation agent computes a sparse BM25 vector for every indexed passage using the `fastembed` library, alongside its dense embedding.

1.3.3.4 Hybrid Search and Reciprocal Rank Fusion

Hybrid search combines dense and sparse retrieval to capture both semantic and lexical relevance. AgriAdvisor's Regulation agent queries Qdrant on both the dense and sparse vectors of every indexed passage simultaneously, then merges the two ranked result lists using **Reciprocal Rank Fusion (RRF)** [14], a rank-aggregation algorithm that combines multiple ranked lists without needing their scores to be on a comparable scale. This hybrid approach is computed server-side by Qdrant and consistently outperforms either retrieval method used alone in retrieval-quality benchmarks [14].

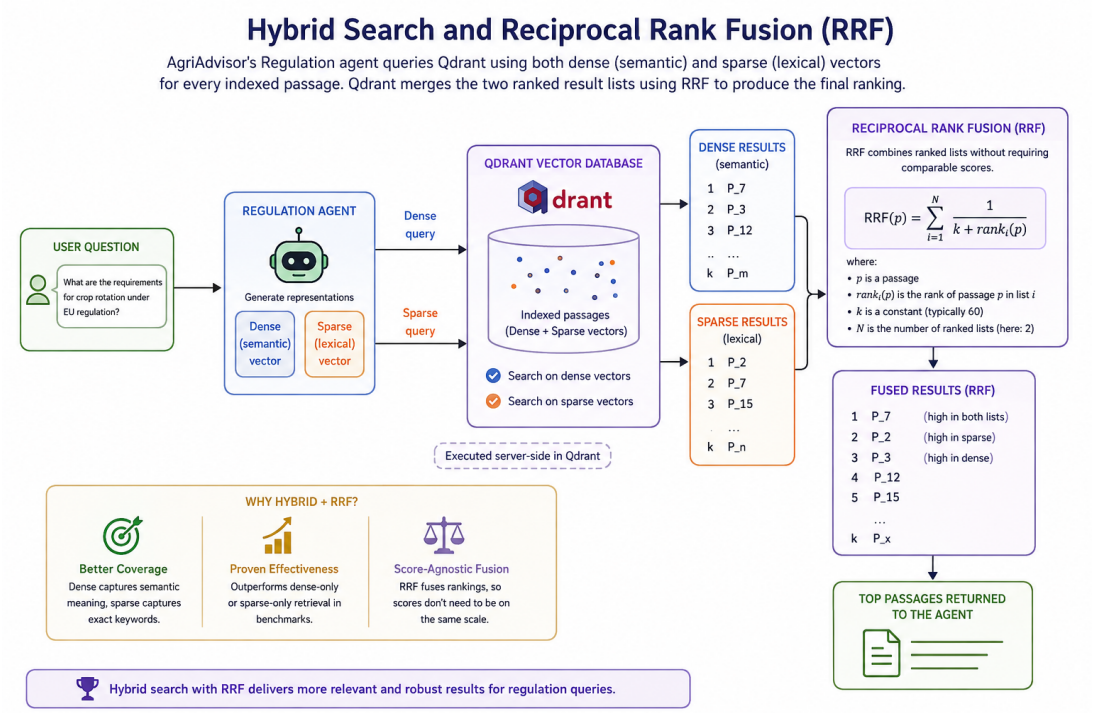

Figure 1.1: Hybrid search combining dense and sparse retrieval

1.3.4 Web Scraping and Automated Corpus Acquisition

1.3.4.1 Definition

Web scraping [15] is the automated extraction of information from web pages by retrieving their content and analyzing their underlying structure. The extracted information can be transformed into structured or semi-structured data for subsequent storage, analysis, or processing. Unlike data acquisition through a dedicated API, web scraping generally operates directly on the content exposed by web pages, most commonly through their HTML documents.

The scraping process typically involves retrieving a web page, identifying the relevant content within its structure, extracting the required elements, and transforming the resulting information into a usable representation. The appropriate technique depends mainly on the structure of the target website and on whether its content is directly available in the initial HTML response or generated dynamically.

1.3.4.2 Web Scraping Techniques

Several techniques can be used to automatically extract information from websites. These techniques differ according to the way web content is accessed, interpreted, and extracted.

Static HTML Scraping. **Static HTML scraping** is used when the desired content is directly contained in the HTML document returned by the web server. The scraper

sends an HTTP request to the target URL, receives the HTML response, and parses the document to locate the required elements. Content can be identified using HTML tags, classes, IDs, attributes, or other structural characteristics. This approach is relatively lightweight because it does not require the execution of client-side JavaScript.

DOM-Based Scraping. The **Document Object Model (DOM)** represents an HTML document as a hierarchical tree of elements. DOM-based scraping uses this tree structure to locate and extract specific elements according to their relationships within the document. Elements can be selected according to their position, parent-child relationships, attributes, or surrounding elements. This approach provides a structured representation of the page and facilitates precise content extraction.

CSS Selector-Based Extraction. CSS selectors provide a concise mechanism for identifying HTML elements according to their tags, classes, IDs, attributes, or combinations of these properties. For example, a selector can target all elements belonging to a particular class or all links contained within a specific section. CSS selectors are widely used in HTML parsing tools because they allow the scraper to specify precisely which elements should be extracted.

XPath-Based Extraction. XPath (XML Path Language) is another technique for navigating and selecting elements from structured HTML or XML documents. XPath expressions describe the path to an element within the document tree and can also express conditions based on attributes, text, or relationships between nodes. XPath is particularly useful when the target information is deeply nested or when more complex selection rules are required.

Browser-Based Scraping. Some websites generate their content dynamically using JavaScript after the initial HTML document has been loaded. In such cases, simply retrieving the HTML source may not provide access to the complete content. **Browser-based scraping** addresses this limitation by using an automated web browser capable of executing JavaScript and rendering the resulting page. This approach can reproduce interactions such as page navigation, button clicks, scrolling, form submission, and dynamic content loading.

Crawling-Based Scraping. When relevant information is distributed across multiple interconnected pages, scraping can be combined with **web crawling**. A crawler begins from one or more starting URLs and follows hyperlinks to discover additional pages. Crawling rules can be used to restrict the process to specific domains, URL patterns, page types, or traversal depths. The combination of crawling and scraping therefore enables the systematic acquisition of information from large collections of web pages.

API-Based Data Acquisition. **API-based acquisition** provides an alternative to HTML scraping when a website exposes a machine-readable interface. Instead of parsing web pages, the client sends requests to predefined API endpoints and receives structured responses, commonly in JSON or XML format. Although API access is technically distinct from traditional web scraping, it is frequently considered within automated web data acquisition because it provides a programmatic mechanism for collecting online information. Compared with HTML scraping, API-based acquisition generally provides more structured and stable data when an appropriate API is available.

Structured Data Extraction. Web pages may contain information explicitly represented in structured formats, including HTML tables, lists, metadata, or embedded structured data such as JSON-LD. **Structured data extraction** focuses on identifying these predefined structures and converting them into organized records. This approach is particularly suitable when the objective is to construct a dataset with clearly defined fields rather than extract unrestricted textual content.

Hybrid Scraping Approaches. In practice, a single technique is not always sufficient to acquire all relevant information from a website. A **hybrid scraping approach** can combine several methods, such as crawling for page discovery, HTTP requests for static pages, DOM or CSS selector-based extraction for content identification, and browser automation for dynamically generated pages. Such combinations allow scraping systems to adapt to heterogeneous website architectures and different types of content.

Overall, web scraping encompasses a range of techniques rather than a single extraction method. Static HTML parsing is appropriate when content is directly available in the server response, while browser-based approaches are more suitable for JavaScript-driven pages. DOM, CSS selectors, and XPath provide different mechanisms for locating content within a page, whereas crawling extends the process across multiple pages. API-based acquisition and structured data extraction provide alternative mechanisms when machine-readable data is available. The choice of technique therefore depends on the technical characteristics of the source and the nature of the information that must be collected.

1.3.4.3 BeautifulSoup

BeautifulSoup [16] is a Python library for parsing HTML and XML documents into a navigable tree structure, offering a simple API to search and extract elements by tag, class, or attribute. AgriAdvisor uses BeautifulSoup to scrape and structure regulatory and subsidy content from three official French sources: **Legifrance** (the Code Rural et de la Pêche Maritime and PAC-related décrets and arrêtés), **FranceAgriMer** (aid schemes, calls for projects, and filière-specific programs), and **Mes Démarches Agriculture** (administrative procedures for farmers). Each scraped page is split into passage-level chunks and enriched with structured metadata — `category`, `knowledge_type` (e.g. *legal_regulation*,

aide_financiere, *appel_a_projet*), `region`, `article_number`, `source`, `source_url`, and `publication_date` — before being embedded and indexed, alongside their BM25 sparse vectors, into the `agricultural_knowledge` collection queried by the Regulation agent's hybrid search (Section 1.3.3.4).

This scraping pipeline is what keeps the Regulation agent's retrieval corpus current: since French agricultural regulation and subsidy amounts change frequently (e.g. new arrêtés fixing per-campaign aid amounts), the corpus must be periodically re-scraped and re-indexed rather than built once and frozen, reinforcing the grounding rationale introduced in Section 1.3.3.1.

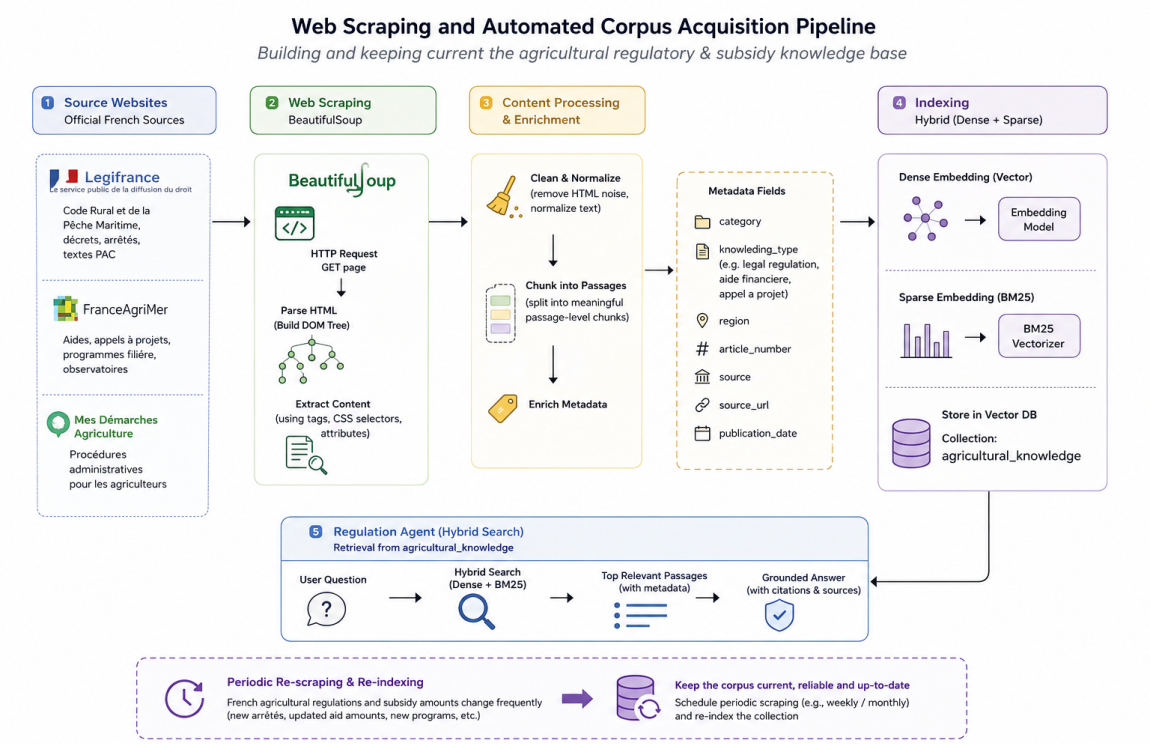

Figure 1.2: Web scraping and automated corpus acquisition pipeline

1.3.4.4 Docling

Docling [17] is an open-source document-conversion library, developed by IBM Research, that parses complex, layout-heavy documents — PDFs, scanned forms, and office formats — into a unified, structured representation that preserves headings, tables, and reading order, rather than the flat, whitespace-mangled text that naive PDF text extraction typically produces. This distinction matters directly for AgriAdvisor's regulatory corpus: while BeautifulSoup scrapes HTML pages from Legifrance, FranceAgriMer, and Mes Démarches Agriculture, a meaningful share of official agricultural documentation — CERFA administrative forms, PDF-published arrêtés and décrets, and aid-scheme annexes listing eligible equipment or per-hectare thresholds — is distributed exclusively as PDF.

AgriAdvisor uses Docling to convert these PDF sources into clean, chunkable text while preserving table structure (e.g., tables of per-crop subsidy amounts or eligible-investment lists, such as those seen in FranceAgriMer's aide dispositifs), so that a figure sitting in a table cell is not silently dropped or scrambled during chunking. The resulting chunks are enriched with the same metadata schema as the BeautifulSoup-scraped content — `category`, `knowledge_type`, `source`, `source_url`, `publication_date` — before being embedded and indexed into the same `agricultural_knowledge` Qdrant collection, so the Regulation agent's hybrid search retrieves from HTML- and PDF-sourced passages indistinguishably.

Using two complementary extraction tools rather than one general-purpose scraper is itself a deliberate design choice: BeautifulSoup is lightweight and sufficient for well-structured HTML, while Docling's layout-aware parsing is reserved for PDFs where structure (particularly tabular data) is load-bearing for correctness — consistent with the broader principle, established for AgriAdvisor's deterministic modeling (Section 1.3.12), that a wrong or dropped number in a regulatory or financial context carries real consequences.

1.3.5 AI-Assisted Frontend Generation

1.3.5.1 Definition

AI-assisted application builders are development platforms that use LLMs to generate and iteratively modify a working frontend codebase from natural-language prompts and visual references, rather than requiring the developer to hand-write every component, layout, and styling rule from scratch. Unlike a low-code/no-code drag-and-drop builder, these tools produce and expose real, editable source code (typically React/TypeScript), so the generated frontend remains a standard codebase that can be further customized, version-controlled, and integrated with an external backend.

1.3.5.2 Lovable

Lovable [18] is one such AI-assisted application-builder, used to develop AgriAdvisor's frontend. Given natural-language descriptions of the desired screens and interactions — the terrain-selection map, the crop-suitability dashboard, the regulation chat interface, the commodity trading tickers and hedging recommendations — Lovable generates the corresponding React components and page structure, which are then connected to AgriAdvisor's backend microservices (Chapter 2, Section 2.4) through their exposed FastAPI endpoints. The generated frontend is built on **React 19** with TypeScript, uses **TanStack Start/Router** for routing, and **TailwindCSS** with **Radix UI** (via shadcn/ui) for the design system — all standard, directly editable code rather than a proprietary format. This approach allowed frontend iteration to proceed in parallel with, and at a pace matching, the rapid evolution of the backend agents' data contracts during development, without requiring a dedicated frontend engineer to hand-build every screen from the outset.

It is worth noting that Lovable, like the frontend it produces, sits outside AgriAdvisor's own AI reasoning pipeline: it is a development-time tool used to build the interface, not a runtime component that participates in generating agronomic, regulatory, or financial recommendations — that reasoning is performed exclusively by the backend agents described throughout this chapter.

1.3.6 3D Geospatial Visualization

3D geospatial visualization is the rendering of terrain, imagery, and spatial data as an interactive three-dimensional scene rather than a flat 2D map, so that elevation, slope, and satellite-derived indices can be inspected visually. AgriAdvisor renders this through two complementary libraries: **Three.js** [19], a JavaScript library for creating and rendering 3D scenes in the browser via WebGL, used to build a parcel-level relief viewer whose mesh is deformed using real elevation data and colored by vegetation or moisture index; and **Mapbox GL JS** [20], a mapping library that renders vector tiles and a global digital elevation model with an adjustable camera tilt, used to navigate the terrain surrounding a selected parcel.

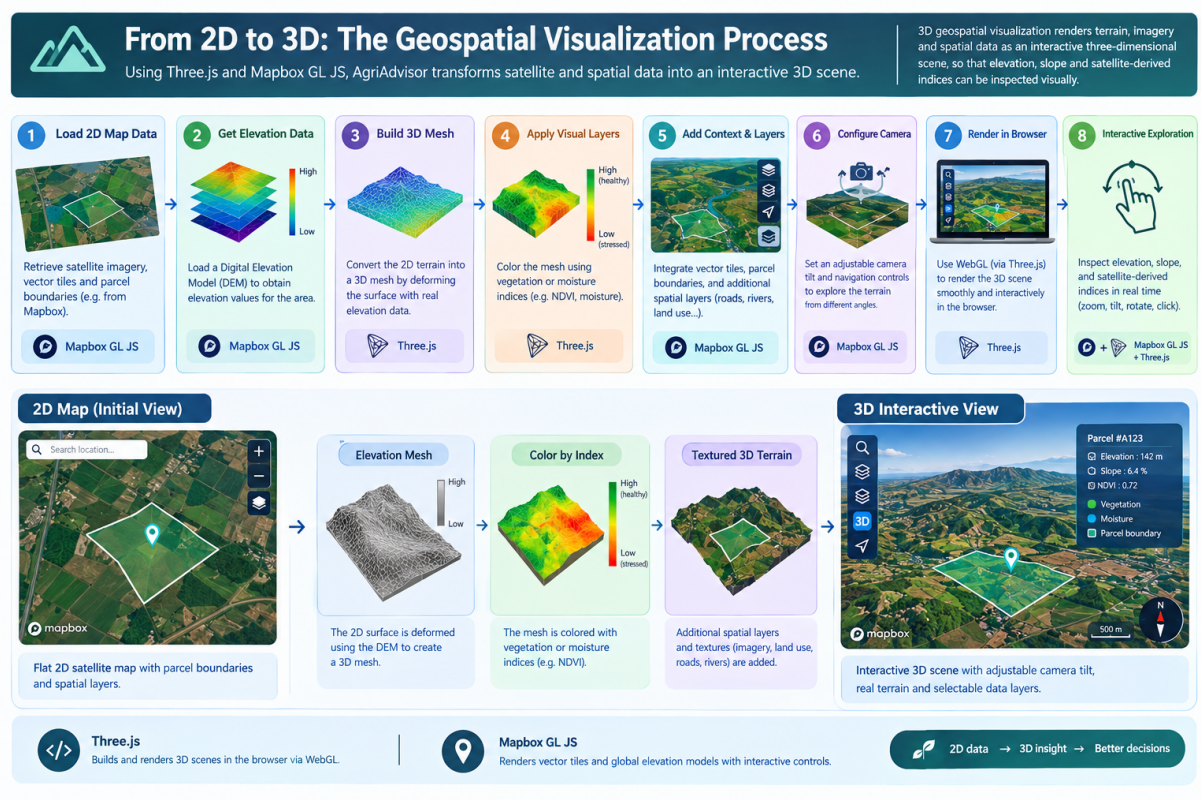

Figure 1.3: From 2D map data to interactive 3D geospatial visualization using Three.js and Mapbox GL JS

1.3.7 Multi-Agent Systems and Orchestration

A **multi-agent system** [21] decomposes a complex task into multiple specialized components — agents — each responsible for a narrower domain, coordinated by an orchestrator or supervisor. This contrasts with a single, monolithic LLM prompted to handle every kind of request: a single-agent design would need one prompt (and one set of tools) to correctly handle regulatory questions, crop suitability, financial scenario planning, monitoring, waste management, and marketplace operations all at once, whereas a multi-agent design lets each concern be engineered, evaluated, and scaled independently.

AgriAdvisor implements this pattern using **LangGraph** [22], a framework for building stateful, graph-structured LLM applications, and **LangChain** [23] for individual agent orchestration. The platform is structured as seven specialized services — an orchestrator/supervisor agent, and dedicated agents for authentication, regulation, agriculture, business, monitoring, and waste — each exposed as an independent FastAPI microservice and communicating over well-defined data contracts (Pydantic schemas).

1.3.7.1 Dynamic Agent Handoff and Control-Shift Routing

A static multi-agent pipeline invokes its specialized agents in a fixed, predetermined sequence regardless of the actual content of the user's request. A **dynamic control-shift architecture** [24], by contrast, allows control to move between agents at runtime based on the evolving content of the conversation, rather than a hard-coded order. This is implemented in LangGraph through two complementary routing mechanisms:

- **Supervisor-based routing**: a central orchestrator node uses an LLM (here, `mistral-small-latest`) to inspect the conversation history, the user's current intent, and the farmer's state (e.g. *onboarding*, *terrain_selectionne*, *analyse_terminee*) and select the next agent to invoke — analogous to the tool-calling pattern described in Section 1.3.2.3, but selecting among *agents* rather than *tools*.
- **Agent self-selection (handoff)**: each specialized agent can itself request a handoff to a different agent once it determines the user's request falls outside its own domain — e.g. the Agriculture agent detecting a subsidy-related keyword and handing off to the Regulation agent — rather than relying exclusively on the supervisor to anticipate every such case.

Both mechanisms are combined with the **human-in-the-loop** routing introduced in Section 1.3.8: when an agent's handoff logic determines that the user's request requires clarification or explicit confirmation before proceeding (e.g. before committing to a financial recommendation), control is routed not to another agent but to human validation, keeping this control-shift mechanism consistent with AgriAdvisor's broader principle that consequential actions require human confirmation rather than autonomous execution.

This design trades the predictability of a fixed pipeline for adaptivity: a query that touches multiple domains (e.g. "is winter wheat still profitable given the new PAC subsidy rules?") can move fluidly between the Business and Regulation agents rather than being forced through agents that have nothing relevant to contribute.

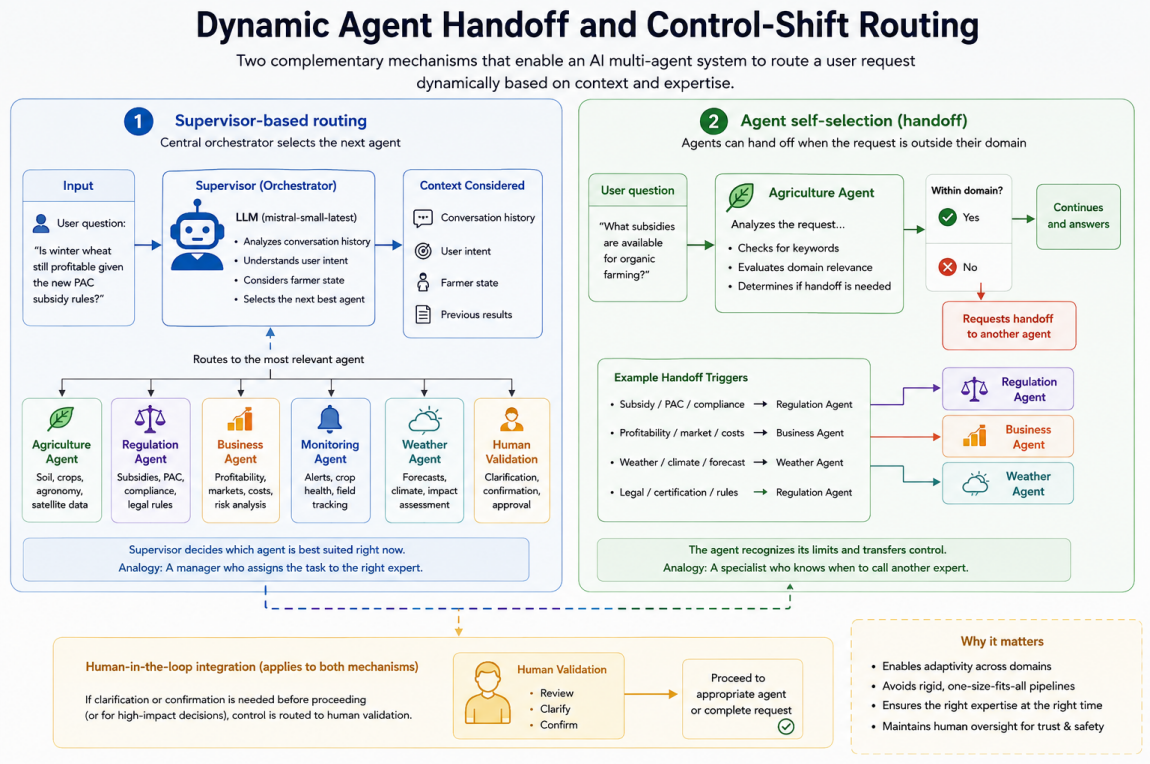

Figure 1.4: Dynamic agent handoff and control-shift routing

1.3.8 Human-in-the-Loop Decision Support

1.3.8.1 Definition

Human-in-the-loop (HITL) [25] is a system design principle in which an AI or automated component produces a recommendation, prediction, or ranked set of options, but a human user retains the authority to validate, override, or reject that output before any consequential action is taken — as opposed to a fully autonomous system that acts directly on its own output.

1.3.8.2 Application in AgriAdvisor

HITL is a deliberate design constraint across AgriAdvisor's higher-stakes agents, most explicitly in the **Trading agent** (Bourse Agricole & Vente à Terme), which evaluates a farmer's market exposure — harvest volume, already-committed tonnage, break-even cost, storage capacity — against live commodity price, trend, and RSI signals, and outputs a `SELL/HOLD/STORE/HEDGE` recommendation together with a tranche-by-tranche

execution plan (immediate sale, limit orders, rally-wait positions) and a confidence score. Crucially, the agent never places an order or signs a forward contract on the farmer's behalf: every recommendation is accompanied by an explicit, human-readable rationale (`rationale`, `step_by_step_plan`) so the farmer can judge and act on it directly, mirroring the auditability principle already established for the Business agent's MCDA scoring. This is a necessary constraint rather than a limitation: an autonomous system committing a farmer's harvest to a forward sale at a bad price, or missing a rally, carries direct financial consequences that must remain under the farmer's control — the same rationale that keeps AgriAdvisor's fertilizer and irrigation calculations deterministic rather than LLM-generated (Section 1.3.12.1).

This complements, rather than duplicates, the grounding principle behind RAG (Section 1.3.3.1): grounding ensures the agent's information is correct and traceable to a source; human-in-the-loop ensures the agent's action remains subject to human judgment even when the information and calculation behind it are sound.

1.3.9 Machine Learning for Crop Suitability

1.3.9.1 RandomForest and Feature-Based Classification

Random Forest [26] is an ensemble machine learning algorithm that builds multiple decision trees on different random subsets of the training data and features, then aggregates their predictions (by majority vote for classification, or averaging for regression). It is widely used for tabular, feature-based classification and regression tasks — including crop recommendation from soil and climate features, where it is consistently reported as the top-performing algorithm among common alternatives (Decision Trees, SVM, KNN, Logistic Regression) in recent literature [27].

AgriAdvisor's Agriculture agent currently implements a documented, rule-based crop suitability scorer as a stand-in for a trained RandomForest classifier. It computes a weighted sum over soil pH, mean temperature, nitrogen content, cation exchange capacity, precipitation, and workability, each scored against per-crop tolerance ranges. The scorer is engineered to produce the exact same output shape a trained `RandomForestClassifier.predict_proba()` call would, so it can later be replaced with a trained model with no changes elsewhere in the pipeline, once labeled training data (e.g., from the French Registre Parcellaire Graphique) is available.

1.3.9.2 Deep Learning for Satellite Time-Series Classification

Beyond tabular scoring, AgriAdvisor incorporates a **Convolutional Neural Network (CNN)** specialized for time-series classification: **TempCNN** [28], a temporal CNN architecture designed specifically for satellite image time-series classification. Unlike recurrent architectures (LSTM) or attention-based Transformers, TempCNN applies convolutions along the temporal dimension of a sequence of multi-spectral satellite observations, and

has been shown to be competitive with, or superior to, these alternatives for crop-type classification from Sentinel-2 time series [29].

Transfer learning [30] is the practice of adapting a model pretrained on one task or dataset to a related task, rather than training from scratch. AgriAdvisor fine-tunes a TempCNN model pretrained on the **BreizhCrops** dataset [29] — a large-scale benchmark of Sentinel-2 time series over Brittany, France, labeled from the RPG — adapting it at a low learning rate over a handful of epochs rather than re-learning temporal patterns from zero. The resulting classifier identifies what is currently growing on a parcel from real satellite observations, used as an independent, real-world evidence signal that lightly (and only when confident) nudges the rule-based suitability score, rather than replacing it.

1.3.10 Remote Sensing and Vegetation Indices

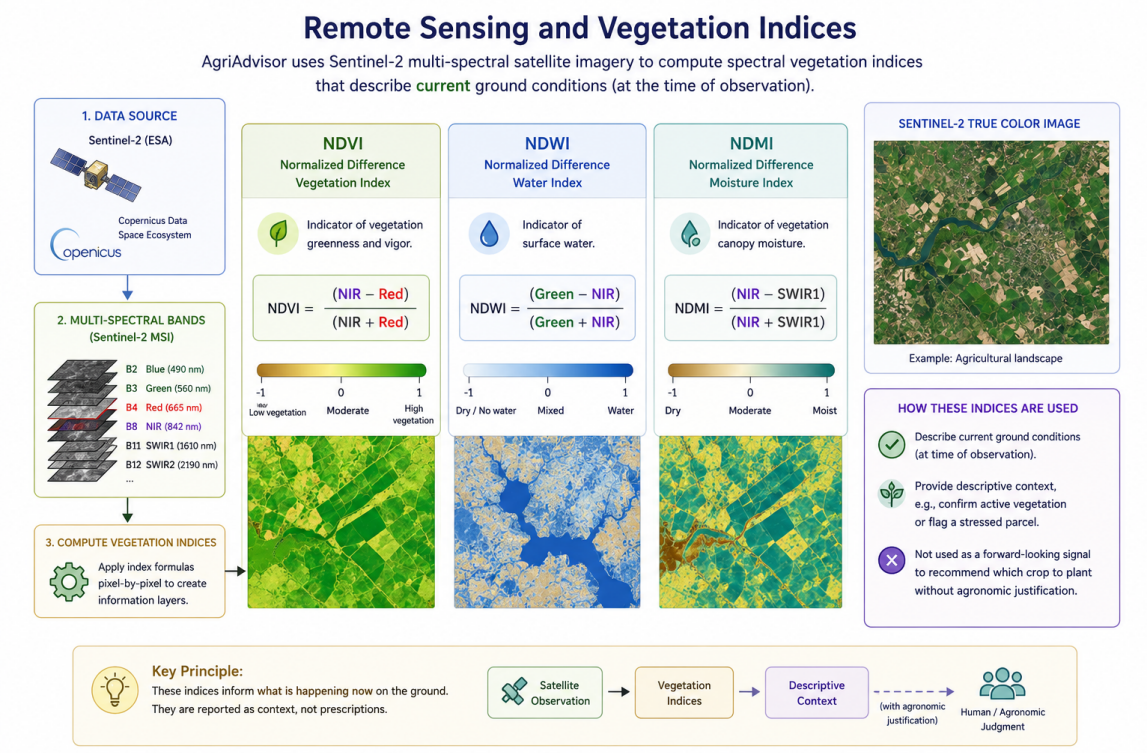

Figure 1.5: Remote sensing and vegetation indices used by AgriAdvisor

Remote sensing refers to the acquisition of information about the Earth's surface via satellite or aerial sensors. AgriAdvisor uses **Sentinel-2** [31] multi-spectral satellite imagery, accessed through the Copernicus Data Space Ecosystem, to compute several **spectral vegetation indices**:

- **NDVI** (Normalized Difference Vegetation Index) [32], computed from the near-infrared and red bands, is a widely used indicator of vegetation greenness and vigor.

- **NDWI** (Normalized Difference Water Index) [33], an indicator of surface water presence.
- **NDMI** (Normalized Difference Moisture Index) [34], an indicator of vegetation canopy moisture.

These indices describe *current* ground conditions at the time of satellite observation — they are reported as descriptive context (e.g., confirming active vegetation or flagging a stressed parcel) rather than treated as a forward-looking signal for which crop should be planted, since doing so without agronomic justification would itself be a form of unfounded inference.

1.3.11 AgriAdvisor's Data Sources

Beyond the retrieval, ML, and remote-sensing techniques described above, AgriAdvisor's decision-support quality depends directly on the breadth and reliability of the external data it consumes. This subsection consolidates the platform's full data inventory, grouped by category, to make explicit what feeds each agent's reasoning.

1.3.11.1 Geographic and Terrain Data

Cadastre IGN / API Carto (Etalab) [35] provides the official French cadastral parcel boundaries, sections, and parcel numbers used to resolve a farmer's selected land into a precise, addressable parcel. The **Registre Parcellaire Graphique (RPG)** [36], published by the IGN via a Web Feature Service (`RPG.LATEST:parcelles_graphiques`), is queried directly to identify the officially declared crop on a given parcel (via its `code_cultu` code, e.g. *BTH* for soft wheat or *MIS* for maize) and its declaration history, and is further used to analyze declared crops in the surrounding area as a proxy for local farming practice. **IGN LiDAR HD** and its associated Digital Terrain Model (MNT) [37] supply high-precision elevation data used to model real parcel relief and compute actual slope, while **BD ORTHO** [38], the IGN's high-resolution aerial orthophotography, is draped over the resulting 3D mesh (Section 1.3.6) alongside **Mapbox GL JS** for global terrain navigation.

1.3.11.2 Satellite and Climate Data

In addition to the Sentinel-2 vegetation indices described above, AgriAdvisor draws on **Open-Meteo** [39] for weather forecasts and historical climate data (temperature, precipitation, and cumulative evapotranspiration) feeding the FAO-56 water balance (Section 1.3.12.1) and the Monitoring agent's alerts, and on **SoilGrids** [40], produced by ISRIC, for global gridded soil properties — clay, sand, and silt content, pH, organic matter, and water retention capacity — used as direct inputs to the Agriculture agent's crop suitability scorer (Section 1.3.9.1).

1.3.11.3 Regulatory and Agronomic Research Data

Alongside the Legifrance, FranceAgriMer, and Mes Démarches Agriculture sources already described (Chapter 2, Section 2.3.1.1), AgriAdvisor's regulatory corpus also incorporates **BCAE** (Bonnes Conditions Agricoles et Environnementales) requirements and éco-régime rules governing PAC eligibility. The Agriculture agent's RAG additionally draws on agronomic research and technical documentation from **HAL**, **ARVALIS**, **Terres Inovia**, and the **ITB** (Institut Technique de la Betterave) — French agricultural research institutes covering cereals, oilseeds, and sugar beet respectively — providing technical guidance beyond what statutory legal text alone can offer. **FAOSTAT** [41], the Food and Agriculture Organization's statistical database, supplies historical per-crop yield series for France (covering soft and durum wheat, maize, barley, sunflower, rapeseed, potato, and tomato, among others), used by the Business agent's scenario generator to compute reference yields, historical trends, and standard deviations that ground its profitability and margin projections. Finally, **Tavily** [42], a search API designed for LLM tool-calling, is used as the Regulation agent's general web-search tool (Section 1.3.2.3) to surface newly published aid schemes and subsidy updates that have not yet been scraped and indexed into the `agricultural_knowledge` collection.

Taken together, these sources mean that AgriAdvisor's agents reason over parcel-accurate geography, real elevation and imagery, live and historical climate, measured soil properties, and both statutory and technical agronomic knowledge — rather than relying on any single data source or on the LLM's own parametric knowledge for any consequential figure.

1.3.12 Deterministic Agronomic and Financial Modeling

Not every decision in AgriAdvisor is made by a trained model or an LLM. Where an incorrect number carries a direct financial or agronomic cost — a fertilizer dose, an irrigation volume, a profitability estimate — the platform instead uses published, deterministic reference formulas, computed in plain code rather than generated by an LLM.

1.3.12.1 COMIFER Nitrogen Balance and FAO-56 Crop Water Balance

The **COMIFER method** [43] is a French reference methodology for computing a crop's nitrogen fertilizer dose, expressed as a balance between the crop's nitrogen needs (scaled to a yield objective) and the nitrogen already supplied by the soil, divided by an apparent utilization coefficient. The **FAO-56 crop water balance** [44] estimates a crop's irrigation need as the difference between crop evapotranspiration (reference evapotranspiration scaled by a crop-specific coefficient, K_c) and effective rainfall. AgriAdvisor implements simplified versions of both methods to estimate fertilizer and irrigation needs per recommended crop.

1.3.12.2 Multi-Criteria Decision Analysis

Multi-criteria decision analysis (MCDA) [45] is a family of methods for evaluating and ranking alternatives against several, often competing, criteria — here, expected profit, risk, budget fit, and agronomic compatibility. AgriAdvisor's Business agent scores each candidate crop scenario as a weighted sum of these four normalized criteria, producing a transparent, auditable *matching score* rather than an opaque model prediction — every component of the score, and the formula that produced it, is exposed to the end user for explainability.

1.4 State of Art

This section reviews research aligned with the three main technical pillars of AgriAdvisor: machine-learning-based crop recommendation, deep-learning-based satellite crop classification, and retrieval-augmented grounding for reducing LLM hallucination in high-stakes domains.

1.4.1 Enhancing Agricultural Productivity: A Machine Learning Approach to Crop Recommendations

This study [27] evaluates nine distinct machine learning algorithms — including Logistic Regression, SVM, KNN, Decision Tree, Random Forest, Bagging, AdaBoost, Gradient Boosting, and Extra Trees — for crop recommendation, using historical data on soil properties, climatic conditions, crop yields, and farmer preferences sourced from the Kaggle repository. The comparison found Random Forest to be the top-performing algorithm, reaching a test accuracy of 99.31%, substantially outperforming weaker baselines such as AdaBoost (14.09%). This confirms Random Forest as a strong, well-supported choice for feature-based crop suitability scoring — the exact algorithm AgriAdvisor's Agriculture agent is designed to adopt once labeled RPG training data becomes available, its current rule-based scorer having been engineered to produce an identical output shape for that future swap.

1.4.2 BreizhCrops: A Time Series Dataset for Crop Type Mapping

This paper [29] introduces BreizhCrops, a large-scale benchmark dataset for the supervised classification of field crops from Sentinel-2 satellite time series, covering the Brittany region of France and labeled from the French Registre Parcellaire Graphique (RPG) — the same registry AgriAdvisor itself queries for parcel resolution and crop history. The authors benchmark seven deep neural network architectures, including TempCNN, LSTM, StarRNN, and Transformer models, against a Random Forest baseline, and release

pretrained model weights publicly. AgriAdvisor's crop-cover classifier is a fine-tuned version of BreizhCrops' pretrained TempCNN checkpoint, used as an independent, real-world observation of what is actually growing on a given parcel — distinct from, and never blended into, the platform's forward-looking suitability scoring.

1.4.3 Reducing Hallucination in Structured Outputs via RAG

This paper [10], published by researchers at ServiceNow, addresses a limitation directly relevant to AgriAdvisor's grounding requirements: LLMs' tendency to hallucinate when producing structured outputs consumed by downstream code, rather than read directly by a human. The authors describe a production enterprise system that uses RAG to significantly reduce hallucination and improve generalization to out-of-domain settings when generating structured workflow representations from natural language. This validates the core design principle behind both AgriAdvisor's Regulation agent (which forces every factual claim to be traceable to a retrieved passage, and forbids introducing figures not present in the retrieved sources) and its Agriculture agent's two-stage synthesis pipeline (which validates every generated claim against an actually-retrieved source chunk, and post-hoc audits every number in the final report against the underlying structured data).

1.4.4 Discussion

Table 1.1 summarizes the reviewed works, their methods, and how they relate to AgriAdvisor's own techniques.

Table 1.1: Summary of Reviewed Work Related to AgriAdvisor's Core Techniques

Paper	**Method**	**Domain**	**Data**	**Relation to AgriAdvisor**
Enhancing Agricultural Productivity: A ML Approach to Crop Recommendations (2024)	Random Forest (best of 9 ML models compared)	Crop recommendation	Soil, climate, yield, farmer preferences (Kaggle)	Target algorithm for the Agriculture agent's suitability scorer (currently a compatible rule-based placeholder)
BreizhCrops: A Time Series Dataset for Crop Type Mapping (2020)	TempCNN (and 6 other DL architectures) vs. Random Forest baseline	Satellite crop-type classification	Sentinel-2 time series, labeled via France's RPG	Source of the pretrained TempCNN checkpoint fine-tuned for AgriAdvisor's DL crop-cover observation
Reducing Hallucination in Structured Outputs via RAG (2024)	RAG for structured-output generation	Enterprise workflow generation	Natural-language requirements	Validates the grounding/citation-validation design of the Regulation and Agriculture agents

While each of these works addresses one facet of AgriAdvisor individually, none combines them: crop-recommendation studies do not address regulatory grounding or satellite time-series observation; BreizhCrops provides a classification benchmark but no downstream decision-support integration; and RAG-hallucination-reduction work is demonstrated in enterprise workflow generation, not in a French agricultural regulatory

or agronomic context. AgriAdvisor's contribution is precisely this integration: a single platform combining hybrid-search RAG grounding for regulation, ML/DL-based crop suitability and satellite observation, and deterministic agronomic and financial modeling, coordinated through a multi-agent architecture and applied specifically to the French agricultural domain.

1.5 Conclusion

This chapter laid the foundation for understanding the context and objectives of the AgriAdvisor project by introducing the key concepts underlying its multi-agent architecture: Large Language Models and tool-calling agents, Retrieval-Augmented Generation and hybrid search, multi-agent orchestration, machine learning and deep learning for crop suitability and satellite observation, and deterministic agronomic and financial modeling. The State of the Art section positioned these techniques against recent, directly relevant research, highlighting that no existing work combines them into a single platform for French agricultural decision support. This sets the stage to present AgriAdvisor's own architecture and contribution in the next chapter.

Chapter 2

Contribution: AgriAdvisor – A Multi-Agent Platform for Agricultural Decision Support

2.1 Introduction

This chapter presents AgriAdvisor, the multi-agent platform developed to automate and support decision-making for French farmers and agricultural buyers. It outlines the methodology used to design and implement the platform – from data acquisition to agent design – details the system's overall architecture and functional requirements, and presents the use case diagram describing how a farmer interacts with the platform's different agents. Together, these elements provide a comprehensive understanding of AgriAdvisor's internal workflow before the next chapter turns to its evaluation.

2.2 Objective of the Contribution

AgriAdvisor was developed to unify, under a single platform, the disconnected sources of information and tools a French farmer would otherwise need to consult separately: legal texts and subsidy portals for regulatory compliance, soil and climate data for crop planning, market statistics and cost estimates for business planning, and commodity exchange quotations for selling decisions. Its core objective is to combine Large Language Models, retrieval-augmented grounding, machine learning, remote sensing, and deterministic financial modeling into a coherent set of specialized agents – each addressing one domain reliably, rather than a single general-purpose assistant attempting all of them at once.

Unlike a generic chatbot, AgriAdvisor grounds every regulatory answer in an indexed, sourced corpus of official texts; scores crop suitability from real soil, climate, and satellite data rather than generic advice; and computes financial and agronomic figures determinis-

tically wherever an invented number would carry a real cost. Developers or farmers using AgriAdvisor do not need to consult Legifrance, FranceAgriMer, SoilGrids, Sentinel Hub, and Euronext/MATIF individually – the platform aggregates and reasons over all of them in one place.

2.3 Methodology

Everything is integrated into a coordinated set of independent services, making the platform easier to extend and maintain, and allowing each agent's data pipeline, grounding strategy, and trust level to be engineered according to its own domain's needs. The development of AgriAdvisor followed a three-phase workflow: first, building the domain-specific knowledge bases and data pipelines each agent relies on; second, designing each agent's reasoning strategy according to how much of its output an LLM may safely generate versus how much must be computed deterministically; and third, selecting and integrating the language and machine learning models used throughout. The following subsections detail each phase.

2.3.1 Data Collection and Knowledge Base Construction

Each agent draws on a distinct set of data sources, collected and structured specifically for its domain.

2.3.1.1 Regulatory Corpus: Web Scraping and Legal Text Structuring

The Regulation agent's knowledge base – the `agricultural_knowledge` Qdrant collection – is built from three scraped sources: **Legifrance**, providing both the full *Code Rural et de la Pêche Maritime* (individual articles, each tagged with its article number and full hierarchical breadcrumb – book, title, chapter, section) and dated *arrêtés* (ministerial orders) setting yearly subsidy amounts (e.g., Politique Agricole Commune payment rates); **FranceAgriMer**, providing structured aid and subsidy program pages, split by section (*Présentation*, *Pour qui ?*, etc.) and paginated when a section exceeds a manageable chunk size; and **Mes Démarches Agriculture**, providing step-by-step administrative procedure pages. Each source is scraped using **BeautifulSoup** [16], a Python library for parsing HTML and extracting structured content from a page's DOM tree, paired with an HTTP client to fetch pages. The scraper extracts not only the article or section text but also its metadata – source, publication date, article number, section title, and, for FranceAgriMer, a `part_index/part_total` pair when a long page is split into multiple chunks – so that every chunk indexed in Qdrant remains traceable to its exact origin, the same citation discipline described for the Regulation agent's RAG tool in Section 2.3.2. This scraping-based ingestion pipeline is distinct from, and complementary to, the agent's live Tavily web search: scraping builds a large, periodically refreshed, pre-indexed corpus of

stable legal and subsidy text, while live web search covers ad hoc, very recent information the indexed corpus does not yet contain.

2.3.1.2 Geospatial and Agronomic Data

The Agriculture agent draws on French cadastral and parcel registries (Etalab API Carto, the Registre Parcellaire Graphique), global soil data (SoilGrids), weather forecasts (Open-Meteo), and Sentinel-2 satellite imagery (Copernicus Data Space Ecosystem), combined with a document corpus of agronomic research papers harvested from HAL, France's national open-access repository.

2.3.1.3 Market and Financial Data

The Business and Trading agents draw on Agreste's IPPAP price-index series, FAOSTAT historical yield statistics, dated market quotations, and, for real-time commodity tracking, the Alpha Vantage financial data API for Euronext/MATIF wheat and corn futures.

2.3.2 Agent Design and Grounding Strategy

A central methodological decision, applied consistently across every agent, is that not all of AgriAdvisor's outputs carry the same trust requirement, and each is engineered accordingly:

- **LLM-generated, strictly grounded:** The Regulation agent's answers and the Agriculture agent's advisory report are produced by an LLM, but only ever from retrieved source passages – every factual claim must trace back to an actually-retrieved chunk, and the system prompt explicitly forbids introducing a figure, date, or reference not present in the retrieved text.
- **ML/DL-predicted:** Crop suitability scores and satellite-based crop-cover classification are produced by a (rule-based, RandomForest-compatible) scorer and a fine-tuned TempCNN classifier respectively – probabilistic outputs, but never generated as free text by an LLM.
- **Deterministic, formula-based:** Fertilizer doses, irrigation needs, yield estimates, production costs, profitability indicators, matching scores, and trading/hedging recommendations are all computed by plain, published-reference formulas, with no LLM involvement at all.

This tiered trust model is why, for example, the Agriculture agent's report-generation pipeline is split into two separate LLM calls (extraction, then writing) with a post-hoc numeric audit in between, and why the Business and Trading agents' scoring and decision-tree logic never calls an LLM at all.

2.3.3 Alignment with the EU AI Act

Because AgriAdvisor provides French farmers with advice that can influence real regulatory compliance, financial, and commercial decisions, its design was considered against the risk-tiered obligations introduced by the **EU AI Act** [46], the European Union's regulatory framework for AI systems, which classifies systems by the severity of harm they could plausibly cause and imposes correspondingly stricter obligations on higher-risk categories. AgriAdvisor does not fall into the Act's small set of *prohibited* practices, and its advisory functions – crop, regulatory, and financial recommendations that a farmer reviews and acts on rather than a system that autonomously executes decisions – place it closer to the Act's *limited-risk* category, for which the central obligation is transparency: users must be able to tell they are interacting with an AI system (Article 50). Rather than treating this as an external constraint layered on afterward, several of AgriAdvisor's design choices already established earlier in this chapter satisfy the spirit of these obligations directly:

- **Transparency and traceability:** The Regulation agent's mandatory source certification (Section 2.3.2) discloses not only that an answer was AI-generated but exactly which official sources it was grounded in – exceeding a bare AI-disclosure requirement with genuine provenance for every factual claim.
- **Human oversight:** The Business agent's requirement that the farmer confirm their final hectare allocation, and the Trading agent's refusal to execute any forward sale or hedging position on the farmer's behalf (Chapter 1, Section 1.3.8), mirror the human-oversight expectations the Act places on systems whose output could materially affect a person's financial position – AgriAdvisor never allows a recommendation to become an action without the farmer's own decision in between.
- **Auditability:** The Business agent's fully exposed MCDA scoring breakdown, and the deterministic COMIFER/FAO-56 calculations underlying fertilizer and irrigation figures, give the farmer – and, if required, a regulator – a fully inspectable calculation rather than an opaque model output, consistent with the Act's broader emphasis on explainability for systems used in consequential decision contexts.

This alignment is a consequence of AgriAdvisor's tiered trust model (LLM-grounded, ML/DL-predicted, deterministic) rather than a retrofit. Because every non-deterministic output is either strictly grounded in a cited source or flagged as a probabilistic prediction, and because every consequential action requires explicit farmer confirmation, the platform's core architecture already satisfies the transparency and oversight principles the AI Act formalizes. This reinforces that these were the right engineering choices independent of the regulatory context that additionally motivates them.

2.3.4 Model Selection

Mistral AI's model family was selected across the platform for its strong French-language performance and its OpenAI-compatible API: `mistral-small-latest` for fast tool-selection decisions, `mistral-large-latest` and `mistral-medium-latest` for final answer and report synthesis, and `mistral-embed` for dense embeddings feeding both the Regulation agent's Qdrant collection and the Agriculture agent's Chroma corpus. For satellite-based crop-cover classification, a TempCNN architecture pretrained on the BreizhCrops dataset was fine-tuned on the platform's own parcel data, rather than trained from scratch, for the reasons detailed in Section 1.3.9.2.

2.4 Platform Architecture and Functional Requirements

2.4.1 Architectural Overview

AgriAdvisor is structured as a set of independent microservices, coordinated by a supervisor/orchestrator agent (Figure 2.1). Each service is implemented as a FastAPI application with its own database access, external API integrations, and, where relevant, its own LLM or ML model, communicating over well-defined Pydantic data contracts rather than shared internal state.

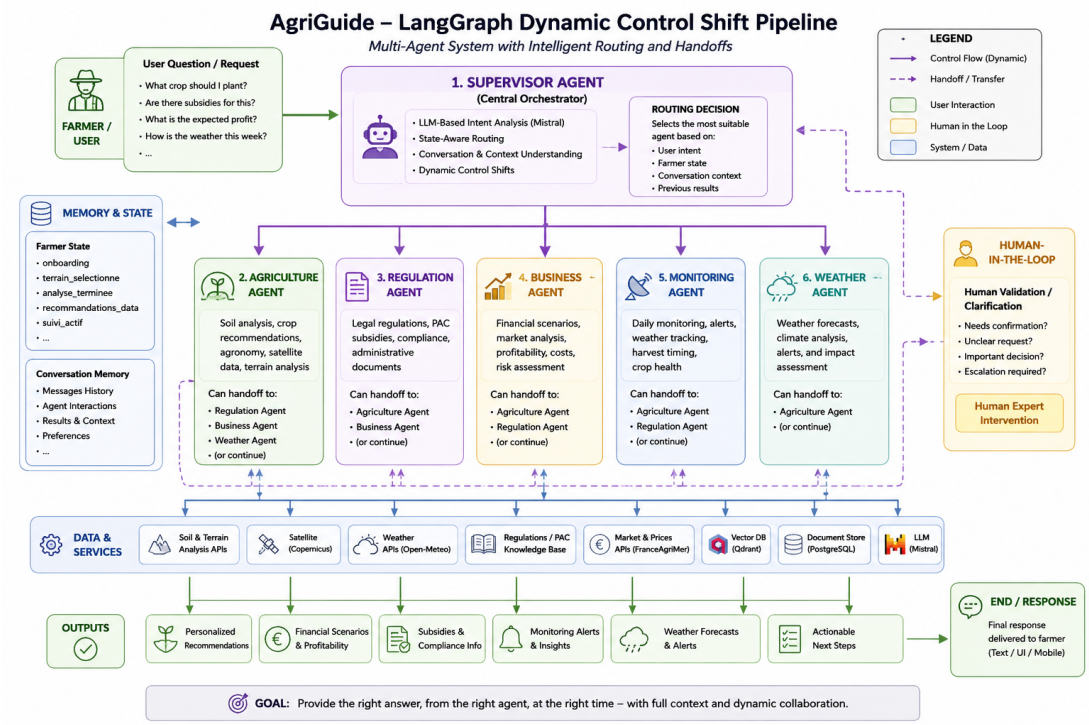

Figure 2.1: AgriAdvisor Platform Architecture

Table 2.1: AgriAdvisor's Microservice Agents

Agent	**Role**
orchestrator	Supervisor agent – routes user requests across agents
auth	User accounts (farmer / buyer roles), terrain ownership
agent_regulation	Legal and subsidy RAG (hybrid Qdrant search + Mistral AI)
agent_agriculture	Parcel resolution, soil/climate/satellite analysis, crop scoring, DL crop-cover classification, agronomic report generation
agent_business	Business scenario generation, financial scoring, risk assessment, human-in-the-loop decision confirmation
agent_monitoring	Daily crop monitoring and automated alerts
waste_agents	Agricultural waste knowledge base and valorization
marketplace	CRUD for harvest and waste listings
agent_trading	Commodity market tickers, technical analysis, hedging and commercialization strategy (SELL/HOLD/STORE/HEDGE)

The frontend (React/TypeScript, built with Vite and TanStack, and the Lovable platform) communicates with the orchestrator, which routes each user request to the relevant agent(s). Agents persist data in a shared PostgreSQL database (with PostGIS and pgvector extensions for geospatial and vector data) and, where applicable, query dedicated vector stores (Qdrant for the Regulation agent, Chroma for the Agriculture agent's document corpus). The whole platform is containerized via Docker Compose, with Cloudflare Workers handling parts of the deployment infrastructure.

2.4.2 Platform Functional Requirements

The platform's core functional requirements, grouped by agent, are:

1. **Regulatory Assistance (agent_regulation)**
    - Answers natural-language questions on French agricultural regulation and subsidies
    - Selects between indexed regulatory search, targeted subsidy search, and general web search depending on the question
    - Returns answers with an explicit, human-readable source certification
    - Maintains a cached, periodically synchronized table of active subsidies

2. **Crop and Parcel Analysis (agent_agriculture)**

    - Resolves a map point or drawn boundary to an exact cadastral/RPG parcel
    - Retrieves soil, weather, vegetation (NDVI/NDWI/NDMI), and satellite-observed crop-cover data for the parcel
    - Scores and ranks candidate crops by suitability, and estimates their fertilizer, irrigation, and yield needs
    - Generates a grounded, French-language advisory report, and answers follow-up questions via a floating chat assistant

3. **Business Planning (agent_business)**

    - Generates ranked business scenarios (crop, expected profit, risk, financing) from the Agriculture agent's recommendations and the farmer's budget
    - Computes deterministic financial indicators (profit margin, ROI, break-even price/yield) and a composite risk score
    - Records the farmer's final hectare allocation as a confirmed decision (human-in-the-loop)

4. **Commodity Trading and Commercialization (agent_trading)**

    - Displays real-time commodity tickers and historical price charts for major French agricultural futures (wheat, corn, rapeseed, sunflower, barley) and fertilizer inputs
    - Computes technical indicators (RSI, moving averages, support/resistance, trend) per commodity
    - Recommends a SELL / HOLD / STORE / HEDGE commercialization strategy from the farmer's harvest position, break-even cost, and storage capacity
    - Proposes a tranche-based forward-contract hedging schedule and surfaces active market alerts

5. **Monitoring, Waste, and Marketplace**

    - Tracks confirmed crops toward harvest and triggers automated alerts (agent_monitoring)
    - Provides a knowledge base and API for agricultural waste valorization (waste_agents)
    - Supports community listings for harvests and waste exchange (marketplace)

2.4.3 Deterministic Agronomic and Financial Calculation Engine

As established in Section 2.3.2, every figure with direct agronomic or financial consequence — irrigation volume, fertilizer dose, projected yield, crop-suitability score — is computed by plain, published-reference formulas rather than generated by an LLM. This section details the exact coefficients implemented in `agro_calc_service.py`, building on the general FAO-56 and COMIFER methods introduced in Section 1.3.12.1.

2.4.3.1 FAO-56 Crop Water Balance (Irrigation Need)

Crop evapotranspiration is computed as:

$$ET_c = ET_0 \times K_c \tag{2.1}$$

where ET_0 is the daily reference evapotranspiration, computed via the FAO Penman–Monteith equation and supplied by the Open-Meteo API (Section 2.3.1.2), and K_c is the mid-season crop coefficient drawn from the FAO-56 reference tables (Table 2.2).

Table 2.2: Mid-season crop coefficients (K_c) used by the irrigation engine

Crop	K_c
Soft wheat	1.15
Maize	1.20
Rapeseed	1.10
Potato	1.15
Soybean	1.15
Sugar beet	1.20
Pea	1.05

The net irrigation requirement is then the cumulative crop water demand minus effective rainfall:

$$\text{Irrigation} = \max\left(0,\ \sum ET_c - P_{\text{eff}}\right) \tag{2.2}$$

where effective precipitation is derived from gross rainfall using an 80% efficiency coefficient:

$$P_{\text{eff}} = \sum P_{\text{gross}} \times 0.8 \tag{2.3}$$

2.4.3.2 COMIFER Nitrogen Balance (Fertilizer Dose)

The nitrogen fertilizer dose follows the COMIFER predictive balance method:

$$\text{Dose N (kg/ha)} = \max\left(0,\ \frac{\text{Needs} - \text{Soil Supply} - \text{Grassland Credit}}{\text{CAU}}\right) \tag{2.4}$$

Needs are the yield objective (q/ha) scaled by a crop-specific unit requirement b (kg N per quintal), sourced from COMIFER/Arvalis reference tables (Table 2.3).

Table 2.3: Unit nitrogen requirement (b) by crop

Crop	b (kg N / quintal)
Wheat	3.0
Maize	2.2
Rapeseed	7.0

Soil Supply is estimated from SoilGrids organic-carbon content, linearly interpolated between 50 and 110 kg N/ha. **Grassland Credit** accounts for nitrogen released by a recently plowed grassland, detected via the TempCNN crop-cover classifier (Section 1.3.9.2): +40 kg N/ha for permanent grassland, +25 kg N/ha for temporary grassland. **CAU** (Coefficient d'Utilisation Apparente / apparent recovery coefficient) is fixed at 0.70.

A legume exception overrides the balance entirely for nitrogen-fixing crops: symbiotic rhizobial fixation means *no* nitrogen input is recommended for pea (0 kg N/ha), and only a light starter dose for soybean (20 kg N/ha).

2.4.3.3 Regulatory Nitrogen Caps (Directive Nitrates / ZVN)

Independently of the COMIFER calculation, the computed dose is capped at the maximum admissible value in nitrate-vulnerable zones (ZVN), so that no recommendation can exceed the regulatory ceiling regardless of the balance result (Table 2.4).

Table 2.4: Maximum admissible nitrogen dose by crop (Directive Nitrates / ZVN)

Crop	Maximum dose (kg N/ha)
Wheat / Maize / Rapeseed / Potato	220
Barley	180
Sugar beet	160
Sunflower	120

2.4.3.4 Deterministic Yield Estimation

Projected yield is computed from the national reference yield (Agreste / FAOSTAT, Section 2.3.1.3), modulated by the crop's agronomic suitability score:

$$\begin{aligned} \text{Yield (q/ha)} = \text{Reference Yield} \\ \times (0.6 + 0.8 \times \text{Suitability Score}) \end{aligned} \tag{2.5}$$

The multiplier is bounded between 60% and 140% of the reference yield, and the final estimate carries an explicit ±15% uncertainty band.

2.4.3.5 Multi-Criteria Crop Suitability Scoring

Extending the rule-based scorer introduced in Section 1.3.9.1, each crop's suitability score is a weighted sum of six soil and climate criteria (Table 2.5), each individually normalized against the crop's agronomic tolerance range before weighting.

Table 2.5: Weighting of the crop-suitability scorer's soil and climate criteria

Criterion	Weight
Soil pH	25%
Mean temperature	25%
Total soil nitrogen	15%
Cation exchange capacity (CEC)	15%
16-day precipitation	10%
Soil workability (stoniness / bulk density)	10%

2.4.4 Agro-Security and LLM Safety Guardrails (AgroLLM-Sec)

Independently of the tiered trust model governing output correctness (Section 2.3.2), AgriAdvisor's LLM-facing agents are wrapped in a dual-interceptor safety layer, AgroLLM-Sec, addressing a distinct concern: preventing malicious or physically/legally dangerous exchanges regardless of which trust tier produced the output.

Input Filter – scans incoming prompts before they reach the LLM, using deterministic regex and Unicode-normalization checks (zero added latency) to catch prompt-injection attempts (jailbreaks, system-prompt overrides, Base64/Unicode spoofing) and queries requesting banned substances, cross-referenced against EU Regulation (EC) No 1107/2009's blacklist of prohibited active substances (e.g., Atrazine, Chlorpyrifos, Paraquat).

Output Firewall – intercepts the LLM's drafted response before it reaches the user, extracting any numeric agronomic recommendation and validating it against the same physical and regulatory limits underlying Section 2.4.3: the nitrogen dose ceilings of the Nitrate Directive (91/676/EEC), and the wind-speed spraying ban (> 19 km/h, Arrêté du 12/09/2006). Rather than using a second LLM call to audit the first – slower and itself hallucination-prone – the firewall performs this check with exact formulas, guaranteeing a deterministic, reproducible verdict.

This design keeps AgroLLM-Sec consistent with AgriAdvisor's broader principle (Section 2.3.2) that consequential figures are checked against formulas rather than a second model's judgment.

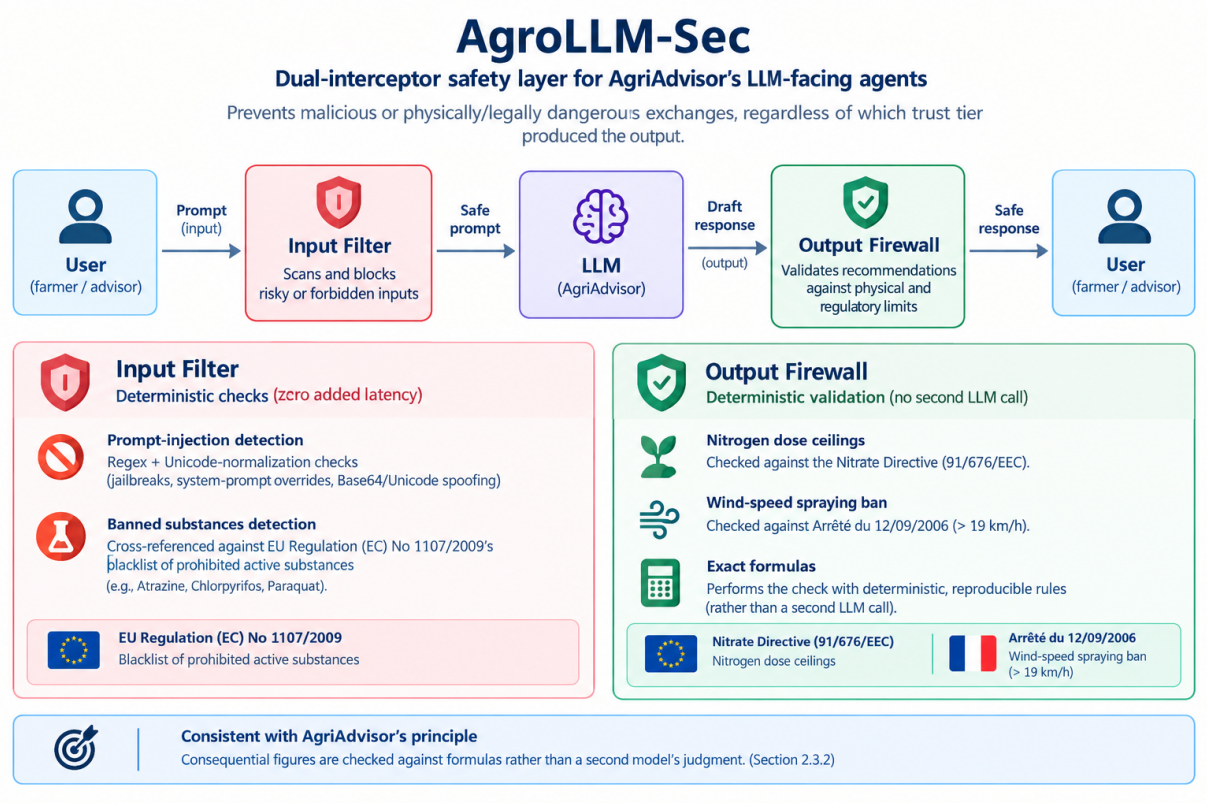

Figure 2.2: AgroLLM-Sec safety layer: input filtering and output validation

2.5 Use Case Diagram

A Use Case Diagram is a type of UML diagram that visually captures the functional behavior of a system from the user's perspective, illustrating the interactions between actors and the use cases they engage with. Figure 2.3 represents the use case model for AgriAdvisor, showing how a farmer interacts with the platform's principal agents.

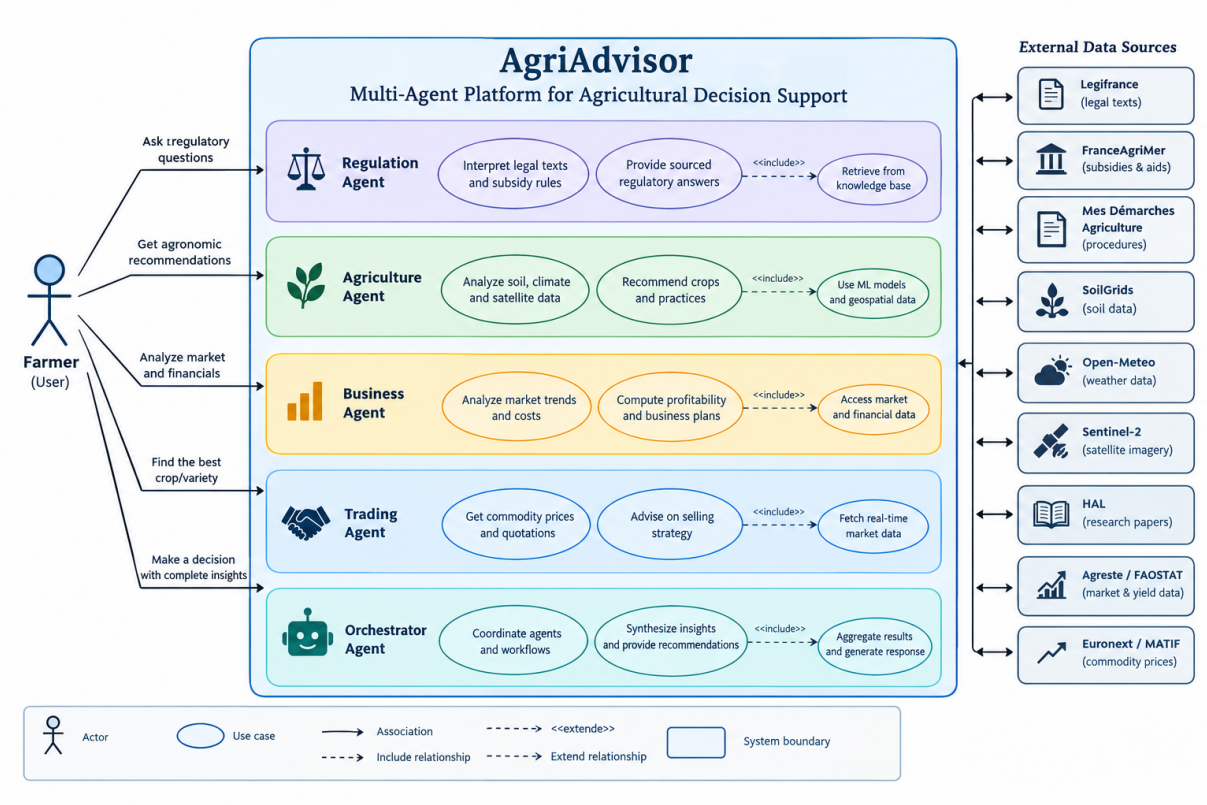

Figure 2.3: AgriAdvisor Use Case Diagram

Use Case Textual Descriptions

The following tables detail the platform's core use cases, one per major agent, in the same structured format used throughout this report.

2.6 Conclusion

This chapter presented AgriAdvisor's contribution in detail: the methodology behind its data pipelines and grounding strategy, its alignment with the EU AI Act's transparency and human-oversight obligations, its multi-agent microservice architecture and functional requirements, and the core use cases through which a farmer interacts with the platform. Together, these elements show how AgriAdvisor combines LLM-based grounding, machine learning, remote sensing, and deterministic modeling into a single, coherent, and regulation-aware platform. The next chapter turns to the evaluation of the models and techniques underlying each agent, justifying the specific choices made throughout the platform.

Table 2.6: Textual description of the "Ask Regulatory Question" use case

Title	Ask Regulatory Question
Principal Actor	Farmer / Buyer
Main Role	The user asks a natural-language question about agricultural regulation or subsidies.
Pre-conditions	The Regulation agent's LLM and Qdrant collection must be available.
Post-conditions	A sourced, French-language answer is returned, with a certification line naming the official sources used.
Nominal Scenario	1. The user submits a question. 2. The agent's LLM selects the appropriate tool(s): indexed regulatory search first, then subsidy search if relevant, then general web search only if needed. 3. Retrieved passages are returned to the LLM, which drafts a sourced answer. 4. The agent appends a certification line and, where available, direct links to the cited sources.
Alternative Scenarios	A1: The indexed corpus already fully answers the question – the web search tool is not called.
Error Scenarios	E1: The Qdrant collection is unreachable – the agent falls back to answering from general knowledge, explicitly disclosing that the regulatory database could not be consulted.

Table 2.7: Textual description of the "Analyze Parcel and Recommend Crops" use case

Title	Analyze Parcel and Recommend Crops
Principal Actor	Farmer
Main Role	The farmer selects a parcel and requests a full agronomic analysis and crop recommendations.
Pre-conditions	The farmer has either selected a point on the map or drawn a parcel boundary.
Post-conditions	Soil, weather, vegetation, and crop-cover data are retrieved; a ranked list of recommended crops with fertilizer/irrigation/yield estimates is returned; results are persisted if the parcel is a saved terrain.
Nominal Scenario	1. The farmer selects a parcel. 2. The system resolves the exact cadastral/RPG boundary. 3. Soil, weather, satellite vegetation, and DL crop-cover data are fetched in parallel. 4. Candidate crops are scored and ranked; fertilizer, irrigation, and yield estimates are computed for the top candidates. 5. A grounded advisory report is generated and returned alongside the raw data.
Alternative Scenarios	A1: No parcel boundary can be resolved – the farmer is prompted to draw one manually.
Error Scenarios	E1: One or more external data sources (satellite, weather) are unavailable – the corresponding field is marked `unavailable` with an explanatory warning, and the rest of the analysis proceeds.

Table 2.8: Textual description of the "Generate Business Scenarios" use case

Title	Generate Business Scenarios
Principal Actor	Farmer
Main Role	The farmer requests business scenarios for their recommended crops, given a budget and planting date.
Pre-conditions	Crop recommendations from the Agriculture agent, an available budget, and a planned planting date must be provided.
Post-conditions	A ranked list of scenarios (culture, financial indicators, risk, matching score) is returned and persisted.
Nominal Scenario	1. The farmer submits crop recommendations, budget, and planting date. 2. For each candidate crop, the system estimates market price/yield, production cost, and risk. 3. A deterministic matching score is computed per crop from profit, risk, budget fit, and agronomic compatibility. 4. The top-N scenarios are returned, each with a full, auditable calculation breakdown.
Alternative Scenarios	A1: A terrain ID is provided – the authoritative, database-verified surface area overrides the value submitted by the client.
Error Scenarios	E1: The terrain does not belong to the requesting user, or does not exist – the request is rejected.

Table 2.9: Textual description of the "Evaluate Commercialization Strategy" use case

Title	Evaluate Commercialization Strategy
Principal Actor	Farmer
Main Role	The farmer requests a recommended selling strategy for their harvest, given their production cost and market conditions.
Pre-conditions	Total expected harvest volume, break-even production cost, and (optionally) already-committed volume and storage capacity are provided.
Post-conditions	A SELL / HOLD / STORE / HEDGE recommendation is returned with a confidence score, rationale, and a step-by-step action plan.
Nominal Scenario	1. The farmer submits their harvest position and cost data. 2. The system retrieves the current market price, trend, and RSI for the relevant commodity. 3. A rule-based decision engine compares market price against the farmer's target price and evaluates trend, RSI, and storage capacity. 4. A recommended action, volume, target price, and estimated gain are returned, along with a market snapshot.
Alternative Scenarios	A1: The farmer has already committed their entire harvest – the system returns HOLD with no further action recommended.
Error Scenarios	E1: The requested commodity symbol is unknown – the system falls back to a default reference commodity.

Chapter 3

Evaluation and Implementation of AgriAdvisor

3.1 Introduction

In this chapter, we evaluate the models and techniques underlying each of AgriAdvisor's agents, then detail the platform's implementation. Because the platform deliberately mixes three different reliability tiers – LLM-grounded generation, ML/DL prediction, and deterministic formulas (see Section 2.3.2) – no single metric can assess the whole system: each tier is evaluated against the criteria that actually matter for it. We first outline this evaluation methodology and the tools used to carry it out, then present the results obtained so far, starting with the Regulation agent, for which a full quantitative benchmark was run, before covering the Agriculture, Business, and Trading agents. The second half of the chapter covers the platform's implementation: the hardware and software environment, the technology stack per agent, the graphical user interfaces, and how the nine microservices are deployed together.

3.2 Evaluation Methodology and Criteria

Unlike a single-model system, where one metric (e.g., accuracy) can rank every configuration, AgriAdvisor's tiered design requires a distinct evaluation approach per tier:

- **LLM-grounded outputs** (Regulation agent answers, Agriculture agent reports) are evaluated on *groundedness* – whether every factual claim traces back to an actually-retrieved source –, on *retrieval quality*, since a grounded answer is only as good as the passages it is grounded in, and against a *no-RAG baseline*, to isolate what grounding actually buys over the underlying LLM's parametric knowledge alone. Tool-selection accuracy is a further criterion identified for this tier (Section 3.3.8) but was not part of the current benchmark run.

- **ML/DL predictions** (crop suitability scoring, satellite crop-cover classification) are evaluated on standard classification metrics – accuracy, Cohen's kappa, and macro/weighted F1 – following the same protocol used to benchmark the pretrained BreizhCrops checkpoint [29] the platform's classifier was fine-tuned from.
- **Deterministic formulas** (fertilizer/irrigation estimates, financial indicators, trading signals) are evaluated on *correctness against the published reference method* (COMIFER, FAO-56) and on *sensitivity* to their configurable weights and unit costs, since there is no "ground truth" to score against beyond the formula itself.

The LLM-related metrics for the Regulation agent were computed using **RAGAS** [47], an open-source evaluation framework purpose-built for RAG pipelines, which provides validated implementations of context precision, context recall, faithfulness, and answer relevancy rather than requiring these to be re-implemented from scratch. RAGAS was configured to run entirely on **Mistral AI**, using `open-mistral-7b` as the judge model and `mistral-embed` for embeddings, each wrapped in a `LangchainLLMWrapper`/`Langchain EmbeddingsWrapper` so that RAGAS's metrics could run against the platform's existing Mistral API plan without requiring a separate OpenAI key. Exponential-backoff retries were added around these calls to absorb rate-limit (HTTP 429) responses and transient network errors during the evaluation run. Alongside RAGAS's built-in metrics, a custom **Document Hit@5** metric was implemented directly, since the available ground truth was a single authoritative `document_id` per question rather than a graded relevance ranking over chunks; this metric checks whether that document is present among the top five retrieved passages. Of RAGAS's metrics, faithfulness and context precision were computed for the Regulation agent benchmark; answer relevancy and context recall are supported by the same harness but were not part of this run. Classification metrics for the Agriculture agent's classifier were computed with **scikit-learn** [48]; rank-stability analysis for the Business agent's weights, with **SciPy**'s [49] implementation of Kendall's rank correlation coefficient, and deterministic-formula correctness, with a dedicated **pytest** [50] test suite comparing implementation output against manually computed reference values, remain planned rather than executed (Section 3.6).

3.3 Evaluation of the Regulation Agent

3.3.1 Gold-Standard Question Set

A gold-standard set of 15 representative questions was curated against the `agricultural_knowledge` Qdrant collection, covering key agricultural regulation and subsidy topics (the 2024–2026 Common Agricultural Policy, FranceAgriMer, Légifrance, and Télépac). Each question was manually paired with the `document_id` of its authoritative source as indexed in Qdrant (for example, `LEGITEXT000051257218_28-02-2025` for leguminous-

crop subsidies) and with ground-truth text extracted directly from the corresponding indexed chunks, so that the benchmark's expected answers stay aligned with what the retrieval pipeline can actually return. A dedicated out-of-scope subset and expected-tool annotations were considered for this gold-standard set but were not included in the present iteration; both are noted in Section 3.6 as directions for extending the benchmark.

3.3.2 Retrieval Configuration Comparison

Four configurations were compared over the indexed collection: a **baseline** with no retrieval at all (the underlying LLM answering from parametric knowledge only), **dense-only** search (`mistral-embed` vectors in Qdrant), **BM25-only** sparse search, and the **hybrid** configuration actually deployed in production (dense and sparse fused server-side via Reciprocal Rank Fusion, Section 1.2.3). For each retrieval-based configuration and each question in the gold-standard set, **Document Hit@5** was computed – whether the question's authoritative `document_id` appears among the top five retrieved chunks – alongside RAGAS's **Context Precision**, which scores the proportion of retrieved chunks that are actually relevant to the question. The baseline configuration retrieves nothing and is therefore not scored on these two metrics; its role is to anchor the faithfulness comparison in Section 3.3.7.

Table 3.1: Regulation Agent – Retrieval Configuration Comparison

Configuration	**Document Hit@5**	**Context Precision**
Baseline (no retrieval)	N/A	N/A
Dense only (`mistral-embed`)	66.7%	0.887
BM25 only (sparse keywords)	73.3%	0.906
Hybrid (Dense + BM25 + RRF, deployed)	66.7%	0.941

BM25-only retrieval obtained the highest raw Document Hit@5 rate, which is consistent with the gold-standard questions frequently naming specific decrees, article numbers, or scheme names (e.g., "BCAE 8", "D614-105") that keyword matching is well suited to. Dense-only and hybrid retrieval tied on Hit@5, but hybrid retrieval achieved the highest Context Precision by a clear margin, indicating that RRF fusion is effective at demoting the noisier passages that dense-only and BM25-only retrieval each let through on their own.

3.3.3 Multi-Rank Retrieval Precision (Hit@k)

A single cutoff can hide where a configuration's retrieval quality actually breaks down, so Document Hit@k was additionally computed at $k \in \{1, 3, 5, 10\}$ for each retrieval configuration, tracking the rank at which the authoritative document is first found (or is never found within the top 10).

Table 3.2: Regulation Agent – Multi-Rank Retrieval Precision (Hit@k)

Configuration	**Hit@1**	**Hit@3**	**Hit@5**	**Hit@10**
Dense only	46.7%	60.0%	66.7%	66.7%
BM25 only	60.0%	73.3%	73.3%	73.3%
Hybrid (Dense + BM25 + RRF, deployed)	60.0%	66.7%	66.7%	73.3%

The three configurations plateau at different ranks. BM25-only reaches its ceiling of 73.3% as early as $k = 3$ and does not improve further, consistent with keyword matching either finding the exact decree or article reference immediately or not finding it at all regardless of how deep the ranking is extended. Dense-only plateaus lower, at 66.7%, from $k = 5$ onward, suggesting a fixed subset of questions – typically those hinging on an exact legal reference rather than a paraphrasable concept – that semantic similarity alone cannot resolve at any depth. Hybrid retrieval trails BM25-only at $k = 1$ and $k = 3$ but is the only configuration that keeps gaining between $k = 5$ and $k = 10$, matching BM25-only's ceiling of 73.3% by $k = 10$ where dense-only remains stuck at 66.7%. This pattern indicates hybrid retrieval's practical value lies less in beating BM25-only outright and more in inheriting BM25-only's exact-match strength while avoiding dense-only's hard plateau – i.e., resilience to either channel's individual failure mode, rather than a strict per-question improvement over the better of the two.

3.3.4 End-to-End Latency Breakdown

To quantify the practical cost of hybrid retrieval's extra fusion step, average end-to-end response time was decomposed into its retrieval and generation components for each configuration.

Table 3.3: Regulation Agent – End-to-End Latency Breakdown

Configuration	**Retrieval**	**Generation (LLM)**	**End-to-End**
Baseline (no retrieval)	0.00 s	3.10 s	3.10 s
BM25 only	0.17 s	3.01 s	3.18 s
Dense only	0.57 s	3.12 s	3.69 s
Hybrid (Dense + BM25 + RRF, deployed)	0.67 s	3.07 s	3.74 s

Generation dominates end-to-end latency across every configuration, at roughly 3–3.1 seconds regardless of what was retrieved. Within the retrieval step itself, BM25-only is fastest (0.17 s), dense-only requires more time for query embedding and vector search (0.57 s), and hybrid retrieval adds only 0.10 s on top of dense-only's cost (0.67 s vs. 0.57 s) for its RRF fusion step. That 0.10 s overhead is small relative to the roughly 3-second generation time it is added to, making RRF fusion's algorithmic cost imperceptible to the

farmer waiting for an answer, and confirming that the Context Precision and Faithfulness gains reported in Sections 3.3.2 and 3.3.7 come at negligible latency cost.

3.3.5 LLM Judge Validation (Human Expert vs. `open-mistral-7b`)

Using a 7-billion-parameter model as the RAGAS judge, rather than a larger model, is a reasonable question for a thesis committee to raise, since a weak or inconsistent judge would undermine every faithfulness and context-precision result reported above. To address this directly, a double-blind validation was run on a representative subsample of 6 questions from the gold-standard set: a human agricultural-law expert scored the faithfulness of each answer independently, without seeing the `open-mistral-7b` judge's scores, and the two sets of scores were then compared.

Table 3.4: LLM Judge Validation – Human Expert vs. `open-mistral-7b` (N=6)

Metric	**Value**
Mean faithfulness – human expert	95.0%
Mean faithfulness – `open-mistral-7b` judge	92.5%
Mean Absolute Error (MAE)	0.025
Pearson correlation (r)	0.980 ($p < 0.001$)

The `open-mistral-7b` judge tracks the human expert's scores almost exactly in relative terms ($r = 0.980$), while running systematically 2.5 percentage points below the human expert on average (92.5% vs. 95.0%), a small conservative bias plausibly explained by the judge being stricter on claims that are correct but expressed as paraphrase rather than a verbatim match to the retrieved passage. Because the near-perfect correlation is what determines whether the judge *ranks* configurations correctly – the property the rest of this chapter's comparisons depend on – rather than whether its absolute faithfulness numbers match a human expert's to the percentage point, this validation supports using `open-mistral-7b` for the relative comparisons in Sections 3.3.2–3.3.7, while the 2.5-point conservative offset should be kept in mind when citing any single configuration's faithfulness score in isolation.

3.3.6 Statistical Significance and Confidence Intervals

With only 15 questions in the gold-standard set, an observed difference between two configurations – such as hybrid's 0.626 faithfulness versus dense-only's 0.593 – could plausibly be sampling noise rather than a genuine effect. Two complementary checks were run to quantify this uncertainty: bootstrap resampling to obtain 95% confidence intervals around each configuration's Hit@5 rate, and a Wilcoxon signed-rank test comparing hybrid retrieval against dense-only retrieval paired at the question level.

Table 3.5: Bootstrap 95% Confidence Intervals – Hit@5

Configuration	**95% CI (Hit@5)**
Dense only	[40.0%, 86.7%]
BM25 only	[53.3%, 93.3%]
Hybrid (deployed)	[46.7%, 86.7%]

The three confidence intervals overlap substantially, and the Wilcoxon signed-rank test comparing hybrid against dense-only on paired per-question faithfulness and Hit@k outcomes returns $p = 0.317$, well above the conventional $p < 0.05$ threshold. Taken at face value, this means the sample of 15 questions is too small to establish hybrid retrieval's superiority over dense-only with conventional statistical confidence. The case for hybrid retrieval made in this chapter therefore rests on the mechanism identified in Section 3.3.3 – that it avoids dense-only's hard plateau at deeper ranks while retaining BM25-only's exact-match strength – and on its consistent lead in Context Precision and Faithfulness (Sections 3.3.2–3.3.7), rather than on a statistically significant Hit@5 gap at this sample size. Reaching conventional significance for the Hit@5 comparison specifically would require extending the gold-standard set to at least $N \approx 50$ questions, which is noted as a direction for future work in Section 3.6.

3.3.7 Groundedness (Faithfulness) Metric and No-RAG Baseline Comparison

For the same question sample, each configuration's generated answer – including the no-RAG baseline's – was scored using RAGAS's *faithfulness* metric, which decomposes an answer into its individual factual claims and verifies, via the LLM judge, whether each claim is entailed by the passages actually retrieved for that question. For the baseline, which retrieves no context by construction, a claim is only counted as faithful when it happens to be factually correct against the same reference sources used for the grounded configurations. The resulting score is the proportion of claims supported out of the total number of claims made – a direct, automated measure of the strict grounding requirement introduced in Section 2.3.2, which forbids the Regulation agent from introducing any figure, date, or reference not present in its retrieved sources.

Table 3.6: Regulation Agent – Faithfulness by Configuration (RAG vs. No-RAG)

Configuration	**Faithfulness (Groundedness)**
Baseline (Pure LLM, no RAG)	0.550
Dense only	0.593
BM25 only	0.621
Hybrid (Dense + BM25 + RRF, deployed)	0.626

Every retrieval-augmented configuration outperforms the no-RAG baseline on faithfulness, and the deployed hybrid configuration achieves the largest gain (+0.076 over baseline), providing the report's primary quantitative evidence for the central claim motivated in Section 2.11.3: that retrieval-augmented generation measurably reduces hallucination relative to relying on an LLM's parametric knowledge alone, in this specific high-stakes regulatory context rather than as a general claim borrowed from the literature. Answer relevancy was not computed for this run; it remains available in the same RAGAS harness for a future iteration of the benchmark.

3.3.8 Tool-Selection Accuracy

Retrieval and groundedness, as evaluated above, assume the agent has already selected the appropriate tool among indexed regulatory search, targeted subsidy search, and general web search (Section 2.2.4). Whether the agent selects that tool correctly in the first place is a distinct failure mode that the current gold-standard set was not annotated to measure, since it does not yet record an expected tool per question. Quantifying tool-selection accuracy, including a confusion breakdown by expected-versus-actual tool, and comparing candidate Mistral generation models (`open-mistral-7b`, `mistral-small-latest`, `mistral-medium-latest`, `mistral-large-latest`) on faithfulness, answer relevancy, latency, and API cost at fixed hybrid retrieval, are both identified as direct extensions of this benchmark rather than results reported here (Section 3.6).

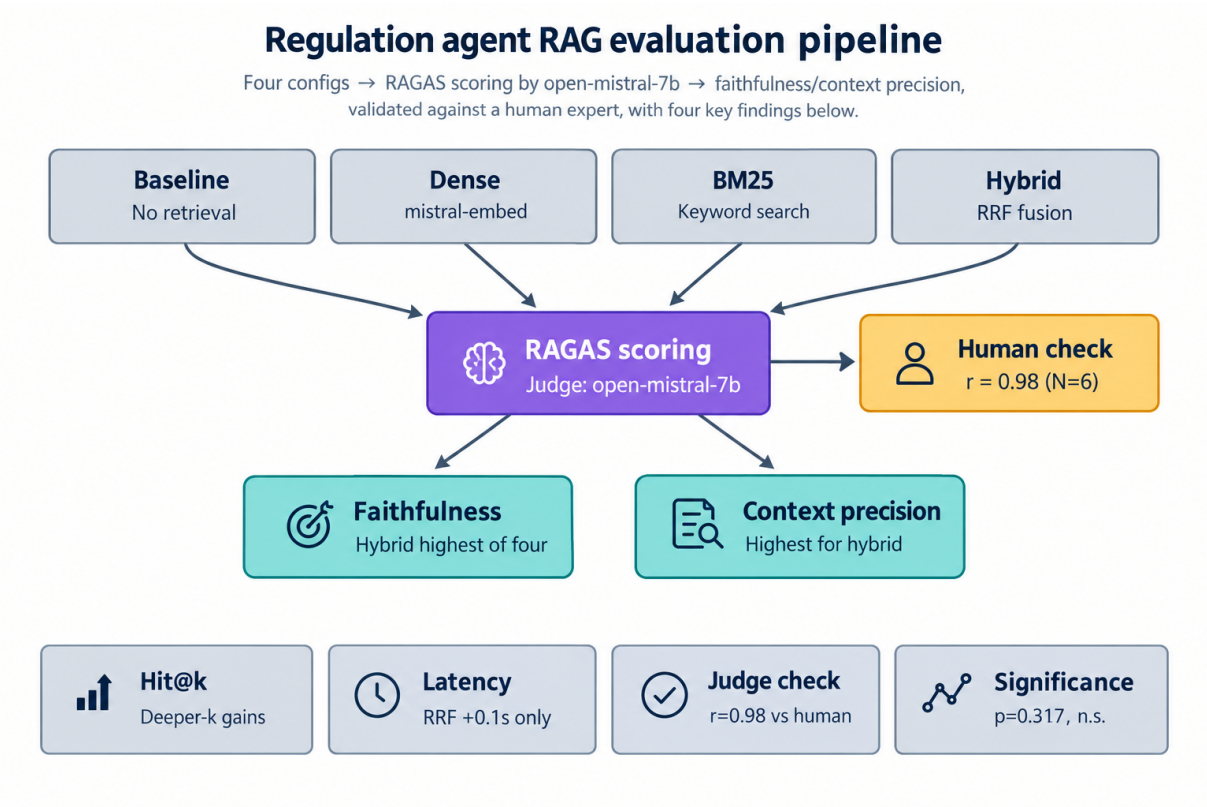

Figure 3.1: Regulation agent RAG evaluation pipeline and key findings

3.3.9 Evaluation of the AgroLLM-Sec Guardrails

AgroLLM-Sec was evaluated with a curated benchmark of 50 test cases across four categories: *prompt injection*, *dangerous agronomic requests* (banned substances), *legitimate edge cases* (legal questions that must *not* be blocked), and *output safety* (dangerous LLM recommendations, e.g. excessive nitrogen doses or spraying during a wind ban).

Table 3.7: AgroLLM-Sec Safety Benchmark Results (N=50)

Metric	Value
Overall safety score	66.6 / 100
Prompt-injection catch rate	53.3%
Dangerous-output catch rate	45.0%
False-positive rate	0.0%

The 0.0% false-positive rate is the most operationally important result: no legitimate agronomic or regulatory question was blocked in this benchmark, meaning the guardrail layer added a safety net without disrupting the tested legitimate requests. The catch rates for prompt injection (53.3%) and dangerous output (45.0%) indicate partial coverage rather than a solved problem, consistent with this being a first-pass deterministic filter rather than a learned classifier; extending the regex/pattern library and the regulatory blacklist is a direct next step, noted alongside the other extensions in Section 3.6 (discussion of evaluation scope and limitations).

3.4 Evaluation of the Agriculture Agent

3.4.1 Crop Suitability Scorer Evaluation

Since no labeled French crop-suitability dataset was available at this project's scope, the rule-based scorer (Section 1.2.6) is evaluated qualitatively against agronomic reference ranges for each of the nine supported crops, and, where the DL crop-cover classifier independently confirms a crop already thriving on a given parcel, the scorer's ranking for that crop is checked for consistency with that observation.

3.4.2 DL Crop-Cover Classifier Evaluation

The fine-tuned TempCNN classifier is evaluated on BreizhCrops' held-out `frh04` region [29], a test split reserved by the benchmark itself and never seen during fine-tuning, using the same metrics as the original benchmark: overall accuracy, Cohen's kappa (which corrects raw accuracy for the agreement expected by chance), and F1 score, reported both macro-averaged, to fairly weight rare classes such as orchards and nuts, and weighted, which favors the platform's more common classes. Results are reported both for the original,

unmodified pretrained checkpoint and for the checkpoint fine-tuned on AgriAdvisor's own parcel data, isolating the effect of that adaptation.

Table 3.8: DL Crop-Cover Classifier – Fine-Tuning Results

Model	**Accuracy**	**Kappa**	**F1 (macro)**	**F1 (weighted)**
Pretrained BreizhCrops checkpoint	0.785	0.718	0.551	0.773
Fine-tuned (this project)	0.803	0.742	0.574	0.797

Fine-tuning improves every metric over the pretrained checkpoint, with the largest relative gain on macro F1 (0.551 to 0.574), consistent with fine-tuning helping disproportionately on the platform's rarer crop classes rather than only on already-well-represented ones.

3.4.3 Report Groundedness and Numeric Audit Evaluation

The Agriculture agent's advisory report pipeline includes a post-hoc numeric audit that flags figures in the generated report that cannot be matched to the underlying structured data within the audit's tolerance thresholds. Its effectiveness was evaluated through a controlled error-injection procedure. Two metrics were computed: **recall**, the proportion of injected errors the audit correctly flags, and **precision**, the proportion of all flagged figures that were genuinely injected errors rather than legitimate source values or rounding-noise perturbations. Precision matters as much as recall here, since an audit that flags every figure indiscriminately would trivially achieve perfect recall while making the report unusable.

Table 3.9: Numeric Audit – Precision and Recall

Metric	**Value**
Recall (existence-based injected errors caught)	1.000
Precision (with plausible rounding-noise false positives)	0.819 (range 0.789–0.857, seeds 42/7/123)
Baseline false-positive rate (clean reports, no noise)	0.000
Recall on semantic-mismatch errors (documented limitation)	0.000

The numeric audit was evaluated with a controlled error-injection procedure across three random seeds (42, 7, 123; 10 synthetic report cases per seed, 3 existence-based errors and 2 plausible rounding-noise perturbations per case). Existence-based injected errors – fabricated figures constructed with a fixed large offset independent of the audit's own tolerance-matching logic – were caught with perfect recall (1.000) across all seeds, confirming that the audit detected all such invented figures in these synthetic cases.

Precision varied across seeds (0.789–0.857, mean 0.819) once plausible rounding-noise artifacts were introduced alongside the injected errors: small perturbations to real source values, comparable in magnitude to independent rounding or reformatting, occasionally exceeded the audit's tolerance thresholds and were flagged as false positives, most often for figures with small absolute magnitude (e.g. pH, NDVI, percentage-based figures) where a given absolute offset represents a larger relative deviation. On clean, unperturbed reports the audit produced zero false positives in every case.

A separate experiment targeting the audit's semantic blind spot – a numerically correct source value attached to the wrong claim – produced 0.000 recall across all seeds, confirming quantitatively that the audit verifies numeric existence in the source data, not semantic attachment to the correct claim. An earlier version of this benchmark, in which injected values were validated against the audit's own matching function before being used as test inputs, was found to make recall and precision mathematically equivalent by construction; the harness was corrected to generate errors independently of the audit's internal logic before these final results were produced.

Figure 3.2: Numeric audit evaluation: error-injection procedure, results, and semantic-mismatch limitation

3.5 Evaluation of the Business and Trading Agents

3.5.1 Financial Calculation Correctness

Since the Business agent's financial indicators (break-even price/yield, ROI, profit margin) and the Trading agent's hedging tranches are computed by deterministic formulas rather than learned or generated, their evaluation is a correctness check rather than a statistical accuracy metric: each formula's output is to be compared, for a set of representative inputs, against a manually computed reference value drawn directly from the published COMIFER and FAO-56 worked examples. A formula will be considered verified when its implementation output matches the reference value within a small, domain-appropriate tolerance (e.g. 0.5 kg N/ha for nitrogen dose, 0.1 mm for irrigation need), confirming the implementation faithfully reproduces the published method rather than introducing calculation drift. This correctness check has not yet been executed against the COMIFER/FAO-56 reference examples (Section 3.6).

3.5.2 Scoring Sensitivity Analysis

The Business agent's matching score (Section 1.2.6) is a weighted sum over four criteria (profit, risk, budget fit, agronomic compatibility). Its sensitivity to the configured weights is to be evaluated by re-ranking the same candidate crop set under several alternative weight configurations (e.g. a profit-weighted and a risk-averse reweighting) and comparing each resulting ranking to the deployed default ($w_1 = 0.35$, $w_2 = 0.20$, $w_3 = 0.20$, $w_4 = 0.25$) using **Kendall's rank correlation coefficient** (τ), a standard measure of ordinal agreement between two rankings that ranges from -1 (fully reversed) to 1 (identical). A τ close to 1 for a given alternative weighting would indicate the crop ranking is largely unaffected by that particular reweighting, confirming the score is not driven disproportionately by any single criterion; a τ noticeably below 1 would indicate the ranking is genuinely sensitive to that criterion's weight, which is itself useful information about which criterion has the most practical influence on the farmer's recommended scenario. This analysis has not yet been executed (Section 3.6).

3.6 Final Configuration Selection

Based on the Regulation agent benchmark, the hybrid dense+sparse retrieval configuration (with RRF fusion) was retained for production despite tying with dense-only retrieval on Document Hit@5 (66.7%) and trailing BM25-only on that same metric (73.3%): hybrid retrieval led on both Context Precision (0.941) and Faithfulness (0.626), the two metrics that most directly capture what the Regulation agent needs – clean, relevant context and a grounded answer – rather than raw document recall alone. This choice is further supported, rather than strictly proven, by the multi-rank analysis of Section 3.3.3, which

shows hybrid retrieval alone avoids both dense-only's plateau at deeper ranks and BM25-only's reliance on exact keyword matches; the Wilcoxon test of Section 3.3.6 indicates this specific advantage does not yet reach statistical significance at the current sample size of 15 questions, and the selection is made on this combined qualitative-and-quantitative basis rather than on a single significant metric. For the Agriculture agent's crop-cover classifier, the fine-tuned TempCNN checkpoint (Section 3.4.2) is retained on the strength of its measured gain over the pretrained baseline on every classification metric. For the Business agent's matching-score weights, the default configuration ($w_1 = 0.35$, $w_2 = 0.20$, $w_3 = 0.20$, $w_4 = 0.25$) remains the platform's design choice, pending the weight-sensitivity results described in Section 3.6. `mistral-small-latest` is used for tool-selection decisions and `mistral-large-latest`/`mistral-medium-latest` for final answer and report synthesis in production – distinct from the `open-mistral-7b` judge model used only within the RAGAS evaluation harness – balancing response latency against answer quality across the platform's agents, though this specific choice of generation model has not itself been benchmarked (Section 3.6). The RAG-versus-no-RAG faithfulness comparison (Section 3.3.7) is retained as the report's primary quantitative evidence that grounding materially reduces hallucination in this high-stakes regulatory context, consistent with the design principle motivated by [10].

The evaluation presented above establishes a full quantitative benchmark for the Regulation agent – retrieval quality (Document Hit@k at four ranks, Context Precision), groundedness (Faithfulness) compared across four configurations including a no-RAG baseline, an end-to-end latency breakdown, a human-vs-LLM judge validation, and a statistical-significance analysis (Sections 3.3.3–3.3.6) – and, for the Agriculture agent, a completed classifier fine-tuning comparison (Section 3.4.2) and a numeric-audit error-injection benchmark (Section 3.4.3).

The significance analysis is the clearest statement of the Regulation agent benchmark's main current limitation: at $N = 15$ questions, hybrid retrieval's advantage over dense-only retrieval on Hit@5 does not reach conventional statistical significance ($p = 0.317$), so the configuration choice in Section 3.6 is justified on Context Precision, Faithfulness, and the multi-rank robustness argument of Section 3.3.3 rather than on a statistically confirmed Hit@5 gap; extending the gold-standard set to roughly 50 questions is the direct way to close this gap.

Beyond sample size, the following remain unexecuted and are noted here as explicit directions for extending the evaluation rather than omitted silently: an answer-generation model comparison across Mistral LLMs (`open-mistral-7b`, `mistral-small-latest`, `mistral-medium-latest`, `mistral-large-latest`) at fixed hybrid retrieval, to quantitatively justify the production choice of generation model; tool-selection accuracy and an out-of-scope refusal rate for the Regulation agent, since the current gold-standard set is not annotated with expected tools or out-of-scope questions; RAGAS's answer-relevancy and context-recall metrics for the retrieval-configuration comparison, which the same harness

supports but which were not computed in this run; the COMIFER/FAO-56 reference-value correctness tests for the Business and Trading agents' deterministic formulas; and the Business agent's weight-sensitivity analysis. It also does not include a systematic multi-turn conversational coherence study or a large-scale user study with real farmers.

The numeric audit's precision degrades under realistic rounding-noise conditions (0.789–0.857 across seeds), meaning an operator reviewing flagged figures should expect some false positives, particularly for small-magnitude values, rather than treating every flag as confirmed invention. Separately, its 0.000 recall on semantic-mismatch errors means a fabricated report could pass undetected by attaching a real number to the wrong claim; extending the audit to check claim-value pairing, not just value existence, is a direct next step. These results come from synthetic report cases and do not establish performance on all real-world reports.

The AgroLLM-Sec benchmark (Section 3.3.9) demonstrates only partial prompt-injection and dangerous-output coverage. Extending the regex/pattern library and the regulatory blacklist is a direct next step; the observed zero false-positive rate applies to the tested legitimate cases and does not establish a zero rate in general use.

3.7 Implementation

3.7.1 Hardware Environment

- **Development machine:** [Processor], [Memory], [Storage], [Operating System]
- **TempCNN fine-tuning environment:** [e.g., Google Colab GPU T4 / local GPU], [VRAM], [Training time]

3.7.2 Software Environment and Tools

3.7.2.1 Backend Stack

- **FastAPI** [51]: a modern, high-performance Python web framework used to implement every one of AgriAdvisor's nine microservices, chosen for its native Pydantic integration (validating every request/response against the platform's data contracts) and asynchronous support.
- **LangChain** [23] and **LangGraph** [22]: used for tool-calling agent orchestration (Regulation agent) and for structuring the platform's multi-agent supervisor pattern.
- **Mistral AI SDK** [6]: the Python client used to call Mistral's chat completion, structured output, and embedding endpoints throughout the platform.
- **PostgreSQL, PostGIS, and pgvector**: the platform's shared relational database, extended with PostGIS for geospatial queries (parcel geometry, centroid computation)

and pgvector where lightweight in-database vector operations are needed, accessed via `psycopg2` with explicit SQL rather than an ORM.

- **Qdrant** [11] and **Chroma** [12]: the vector databases backing the Regulation agent's hybrid search and the Agriculture agent's document corpus, respectively.

3.7.3 Chunking, Embedding, and Indexing Pipeline

To operationalize the corpus-acquisition pipeline described in Section 2.3.1.1 (Figure 1.2), the scraped FranceAgriMer JSON output is converted into standardized, independently retrievable chunks and indexed into Qdrant through four scripts: `chunker.py`, `config.py`, `sparse_embed.py`, and `embed_to_qdrant.py`.

3.7.3.1 Chunk Schema

Each dispositif (aid scheme or call for projects) is split section-by-section (*Présentation*, *Pour qui ?*, *Quand ?*, *Comment ?*), with an additional chunk per associated document (PDF/DOCX annexes). Every chunk is assigned a stable, traceable identifier following the same `{source}_{category}_{document_id}_{chunk_reference}` convention used for the Légifrance corpus:

```
def make_chunk_id(category: str, document_id: str, chunk_reference: str) -> str:
    """
    {source}_{category}_{document_id}_{chunk_reference}
    ex: franceagrimer_AIDE_achat-dengrais-mineraux-azotes-simples_presentation
    """
    parts = [SOURCE, slugify(category, max_len=40), document_id, chunk_reference]
    parts = [p for p in parts if p]
    return "_".join(parts)
```

Listing 3.1: Chunk identifier construction (chunker.py)

Sections exceeding a maximum length (1,800 characters) are further split at paragraph boundaries rather than mid-sentence, and each resulting chunk is enriched with the same metadata schema used across the platform's regulatory corpus (`category`, `knowledge_type`, `region`, `document_id`, `title`, `source`, `source_url`, `publication_date`).

3.7.3.2 Dense and Sparse Embedding

Each chunk is embedded twice: a 1024-dimensional dense vector from Mistral's `mistral-embed` model, computed over the chunk's title and text concatenated together, and a sparse BM25 vector computed locally via `fastembed` (`Qdrant/bm25`), requiring no external API call:

```
def build_embedding_text(chunk):
    """Text actually sent to the dense embedding model: title + content."""
    return f"""
```

```
Titre :
{chunk["metadata"]["title"]}

Texte :
{chunk["text"]}
"""

def embed_sparse(texts: list) -> list:
    """One Qdrant sparse vector per text, in the same order as `texts`."""
    embeddings = _get_model().embed(texts)
    return [
        SparseVector(indices=e.indices.tolist(), values=e.values.tolist())
        for e in embeddings
    ]
```

Listing 3.2: Embedding text construction and sparse embedding (embed_to_qdrant.py, sparse_embed.py)

3.7.3.3 Hybrid Collection Creation and Upsert

The two vector types are stored under named vectors (`text-dense`, `text-sparse`) within a single Qdrant collection (`agricultural_knowledge`), enabling the server-side hybrid search described in Section 1.3.3.4. Point identifiers are derived deterministically from each chunk's `chunk_id` via UUID5, so re-running the pipeline on updated data updates existing points rather than duplicating them:

```
if not client.collection_exists(collection):
    client.create_collection(
        collection,
        vectors_config={
            config.QDRANT_DENSE_VECTOR_NAME: VectorParams(
                size=config.MISTRAL_EMBED_DIM, distance=Distance.COSINE
            )
        },
        sparse_vectors_config={config.QDRANT_SPARSE_VECTOR_NAME: SparseVectorParams()},

    )

def _point_id(chunk_id):
    """Stable Qdrant ID derived from chunk_id: re-running the script updates
    the same points instead of creating duplicates."""
    return str(uuid.uuid5(uuid.NAMESPACE_URL, chunk_id))
```

Listing 3.3: Hybrid collection creation and idempotent point IDs (embed_to_qdrant.py)

Batched upserts are wrapped in an exponential-backoff retry loop, configured through `HTTP_MAX_RETRIES` and `HTTP_BACKOFF_FACTOR`, to absorb transient Qdrant Cloud connectivity errors.

3.7.3.4 Indexing Verification

A standalone verification script, `check_qdrant.py`, confirms the collection's point count and, optionally, runs a live query against the dense vector to sanity-check retrieval quality before the corpus is exposed to the Regulation agent:

```
python check_qdrant.py --query "aide achat engrais"
```

Listing 3.4: Indexing verification (check_qdrant.py)

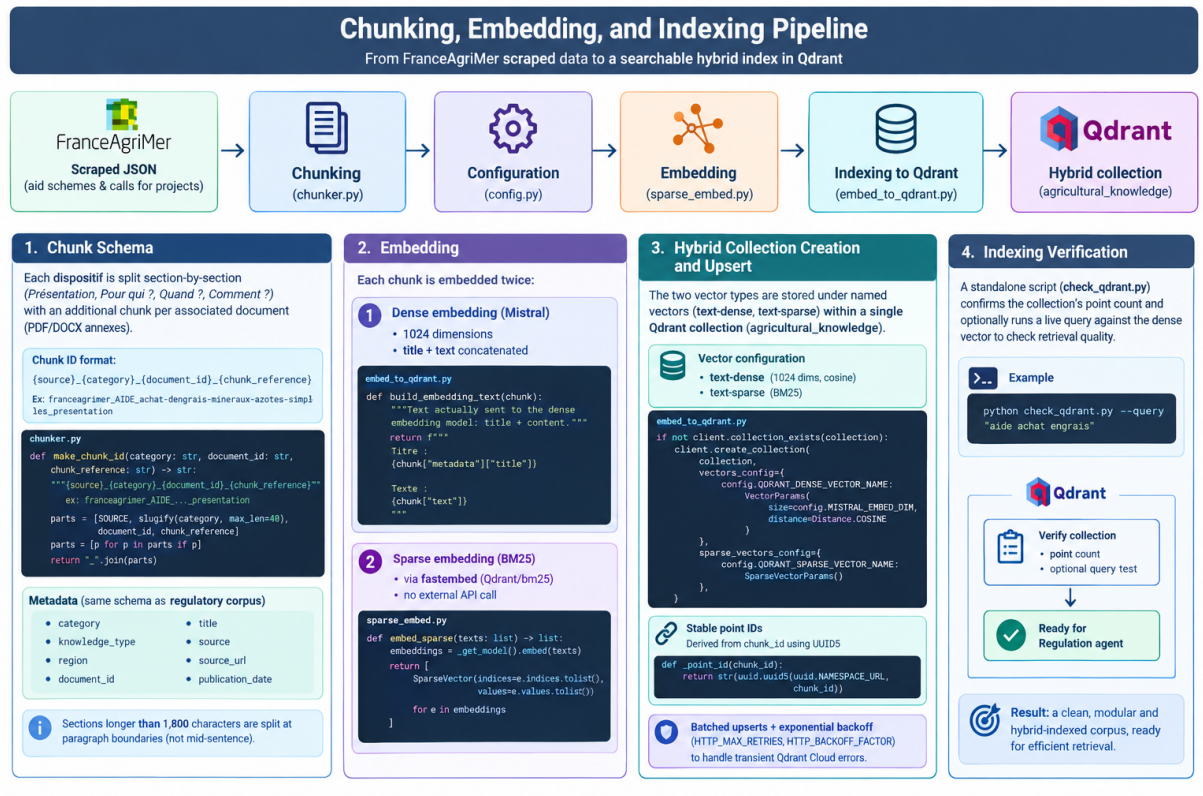

Figure 3.3: Chunking, embedding, and indexing pipeline for the FranceAgriMer corpus

3.7.3.5 Evaluation Stack

- **RAGAS** [47]: provides the faithfulness and context precision metrics used to evaluate the Regulation agent, configured to run on Mistral AI (`open-mistral-7b` as judge, `mistral-embed` for embeddings) via `LangchainLLMWrapper`/`LangchainEmbeddings Wrapper` (Section 3.2); answer relevancy and context recall are supported by the same harness for future benchmark iterations.
- **Custom Document Hit@k metric**: implemented directly alongside RAGAS to check, at $k \in \{1, 3, 5, 10\}$, whether each question's authoritative document was retrieved in the top k passages, with exponential-backoff retries around the underlying API calls.

- **SciPy** [49]: used for the Wilcoxon signed-rank test and bootstrap 95% confidence intervals in the Regulation agent's statistical-significance analysis (Section 3.3.6); Kendall's tau rank-correlation computation for the Business agent's weight sensitivity analysis remains planned.
- **Human-expert double-blind review**: a domain expert independently scored faithfulness on a 6-question subsample, used to validate the `open-mistral-7b` RAGAS judge (Section 3.3.5).
- **scikit-learn** [48]: used for accuracy, Cohen's kappa, and F1 scoring in the TempCNN classifier evaluation (Section 3.4.2).
- **pytest** [50]: planned for reference-value correctness tests of the COMIFER and FAO-56 deterministic formulas.

3.7.3.6 Frontend Stack

- **React** [52] with **TypeScript**, built using **Vite** [53] for fast development builds and **TanStack** [54] for data-fetching and state management, and assembled with the **Lovable** platform.

3.7.3.7 Infrastructure

- **Docker Compose** [55]: orchestrates the platform's nine microservices, shared database, and vector stores as a single local or deployed stack.
- **Cloudflare Workers**: handles parts of the platform's edge deployment infrastructure.

3.7.3.8 Data Acquisition Tools

- **BeautifulSoup** [16]: used to scrape and structure the Regulation agent's Legifrance, FranceAgriMer, and Mes Démarches Agriculture corpus.
- **fastembed**: used to compute BM25 sparse vectors for hybrid search.
- **PyTorch** and **breizhcrops**: used to fine-tune the TempCNN crop-cover classifier.

3.7.3.9 Version Control and Collaboration

- **Git** [56]: used for version control and collaboration across the platform's nine services.

3.8 Graphical User Interfaces

In this section, we present a selection of screenshots of AgriAdvisor's graphical interfaces.

3.8.1 Parcel Analysis Page

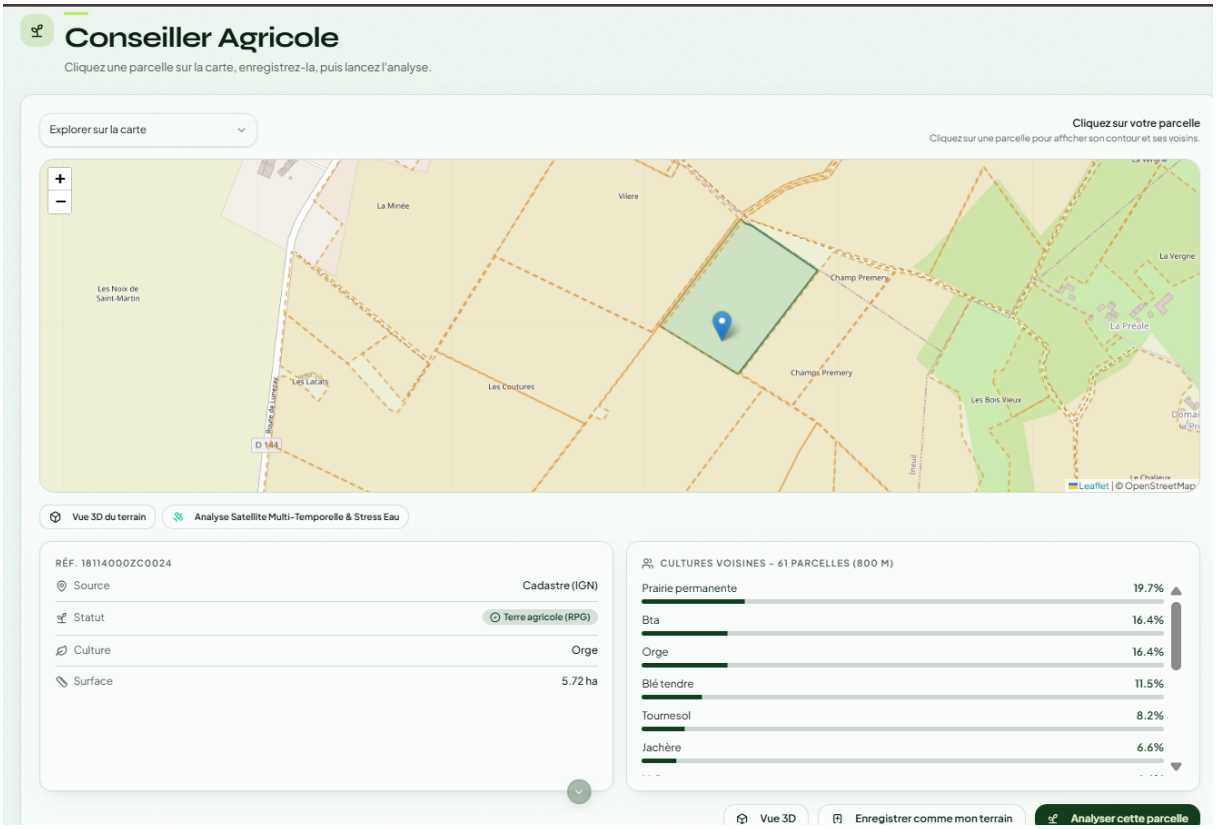

Figure 3.4: Parcel Analysis Page: Map Selection and Crop Recommendations

The parcel analysis page lets the farmer select a point or draw a boundary on the map, triggering the Agriculture agent's full pipeline, and displays the resulting soil, weather, vegetation, and crop recommendation data alongside the generated advisory report.

3.8.2 Regulation Chat Interface

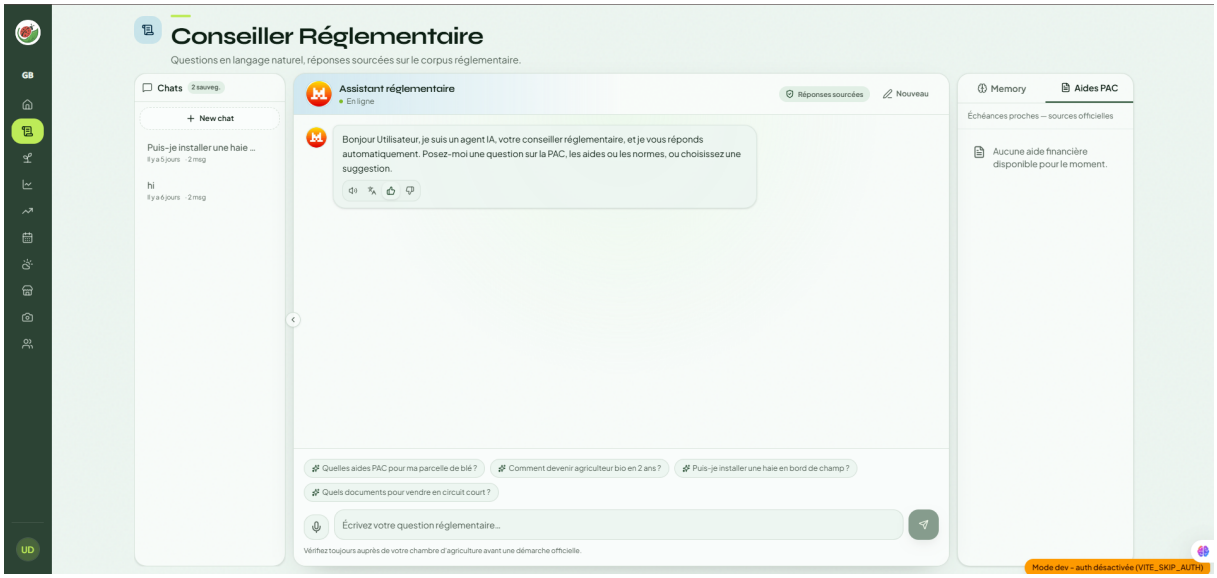

Figure 3.5: Regulation Agent Chat Interface

The regulation chat interface lets the farmer or buyer ask natural-language questions about agricultural regulation and subsidies, displaying the agent's sourced answer alongside clickable links to the cited official texts.

3.8.3 Business Scenarios Page

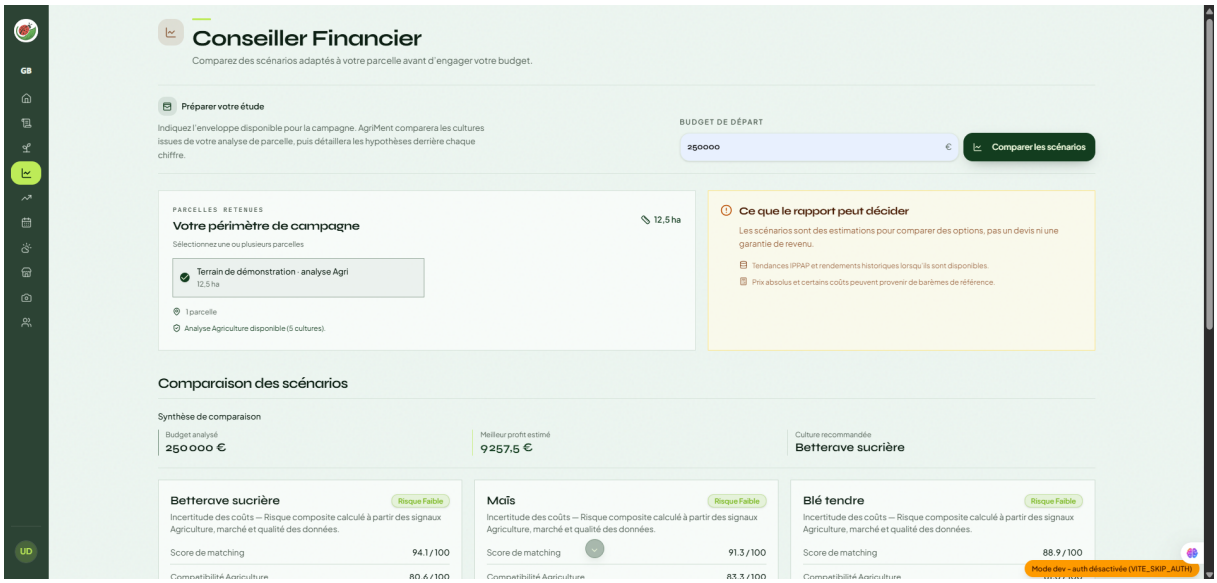

Figure 3.6: Business Scenarios Comparison Page

The business scenarios page presents the ranked crop scenarios returned by the Business agent, each with its financial indicators, risk assessment, and an expandable "Details"

panel showing the full, auditable calculation breakdown.

3.8.4 Trading Dashboard

Figure 3.7: Trading Dashboard: Commodity Tickers and Commercialization Strategy

The trading dashboard displays real-time commodity tickers, technical indicators, and the recommended SELL/HOLD/STORE/HEDGE commercialization strategy for the farmer's harvest position.

3.9 Deployment

AgriAdvisor's nine microservices, PostgreSQL database, and Qdrant/Chroma vector stores are deployed together via Docker Compose, with each service exposed on its own port (Table 2.1) and communicating over internal Docker networking. Environment-specific secrets (API keys for Mistral, Sentinel Hub, Alpha Vantage, and Tavily) are supplied via a shared root `.env` file, with per-service `.env` overrides where a service needs a locally-scoped value. The frontend is built separately and served independently of the backend services, communicating with the orchestrator over its exposed REST API.

3.10 Conclusion

This chapter evaluated AgriAdvisor's core techniques according to each reliability tier. For the Regulation agent, four configurations – no-RAG, dense-only, BM25-only, and hybrid retrieval – were compared using RAGAS and Document Hit@k. Hybrid retrieval achieved the highest context precision and faithfulness, although BM25-only remained competitive on document retrieval. All retrieval-augmented configurations outperformed the no-RAG baseline on faithfulness. Latency analysis showed that fusion adds little overhead compared with LLM generation, while a double-blind expert review supported the judge's reliability. However, the 15-question benchmark limits the strength of the statistical conclusions.

For the Agriculture agent, the fine-tuned TempCNN improved on the pretrained BreizhCrops checkpoint across all classification metrics. The corrected numeric-audit benchmark achieved 1.000 recall for existence-based errors, mean precision of 0.819 under rounding noise, and no false positives on clean reports. Its 0.000 recall on semantic mismatches nevertheless highlights the need to verify claim-value pairing. Reference-value tests for deterministic formulas, Business-agent weight-sensitivity analysis, and further Regulation-agent evaluations – tool selection, out-of-scope refusal, and generation-model comparison – remain future work.

The chapter also presented the hardware and software environment, technology stack, graphical interfaces, and Docker Compose deployment. Together, these results establish a working multi-agent platform with quantitative validation for its Regulation and Agriculture agents, while making the remaining evaluation gaps explicit. They connect the problem identified in Chapter 1 and the design proposed in Chapter 2 to the implemented system.

General conclusion

In conclusion, the development of AgriAdvisor has been both rewarding and technically enriching. This report presented the key phases of our work — from foundational concepts and state of the art to the design, evaluation, and implementation of our multi-agent decision-support platform for French farmers and agricultural buyers.
French farmers face fragmented, fast-changing information across regulation, crop suitability, business planning, and market commercialization, where an outdated figure or unfounded recommendation can carry real consequences. By combining hybrid-search RAG grounding, ML/DL-based crop and satellite analysis, and deterministic agronomic and financial modeling, coordinated through a dynamically-routed multi-agent architecture, we built an integrated platform that keeps every recommendation traceable to a source and every consequential action subject to human validation.

Key strengths of our solution include hallucination-resistant retrieval grounded in official Legifrance and FranceAgriMer sources, transparent and auditable MCDA and COMIFER/FAO-56 calculations, adaptive dynamic agent handoff, and a human-in-the-loop design that keeps decisions like forward-sale hedging under the farmer's control.

Several avenues remain for future work: replacing the rule-based crop scorer with a trained Random Forest classifier once labeled RPG data is available, expanding the TempCNN pipeline beyond Brittany, extending corpus ingestion to PDF-native sources via Docling, and deepening integration between the Business and Trading agents.

Ultimately, AgriAdvisor opens promising perspectives for AI-assisted agricultural decision support in France, with the potential to ease regulatory compliance, improve agronomic and financial decisions, and promote responsible, human-supervised AI assistance across the French agricultural sector.

Bibliography

[1] Talan. Who we are? https://www.talan.com/americas/en/about-us/our-vision/who-we-are. Accessed September 15, 2026.

[2] Talan. Our locations: Tunis excellence center. https://www.talan.com/americas/en/our-locations. Accessed September 15, 2026.

[3] Precision agriculture, 2023. General reference — replace with your preferred precision-agriculture overview source.

[4] Durgaprasad P. Understanding large language models. https://www.linkedin.com/pulse/understanding-large-language-models-durgaprasad-potnuru-rekgc/?trackingId=JriBKPyIQBeZWv83xjxQ1w%3D%3D, 2025. Accessed on April 5, 2025.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[6] Mistral AI Team. Models overview. https://docs.mistral.ai/getting-started/models/models_overview/, 2024.

[7] Structured output generation with large language models, 2024. General reference on constrained/structured LLM output generation — replace with your preferred source.

[8] Tool-calling / function-calling agents, 2024. General reference on LLM tool-use / function-calling agent patterns — replace with your preferred survey or framework documentation.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[10] Patrice Béchard and Orlando Marquez Ayala. Reducing hallucination in structured outputs via retrieval-augmented generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Industry Track)*, pages 228–238, Mexico City, Mexico, 2024. Association for Computational Linguistics.

[11] Qdrant Team. Qdrant: Vector database documentation. `https://qdrant.tech/documentation/`, 2024.

[12] Chroma Team. Chroma: The ai-native open-source embedding database. `https://www.trychroma.com/`, 2024.

[13] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[14] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.

[15] Scrapy developers. Scrapy at a glance. `https://docs.scrapy.org/en/latest/intro/overview.html`. Accessed September 15, 2026.

[16] Leonard Richardson. Beautiful soup documentation. `https://www.crummy.com/software/BeautifulSoup/bs4/doc/`, 2024.

[17] Docling project. Docling documentation. `https://github.com/docling-project/docling/blob/main/docs/index.md`. Accessed September 15, 2026.

[18] Lovable. Welcome to Lovable. `https://docs.lovable.dev/introduction/welcome`. Accessed September 15, 2026.

[19] Three.js contributors. Creating a scene. `https://threejs.org/manual/en/creating-a-scene.html`. Accessed September 15, 2026.

[20] Mapbox. Mapbox GL JS documentation. `https://docs.mapbox.com/mapbox-gl-js/guides/`. Accessed September 15, 2026.

[21] Multi-agent systems with large language models, 2024. General reference on LLM-based multi-agent architectures — replace with your preferred survey.

[22] LangChain Team. Langgraph documentation. `https://langchain-ai.github.io/langgraph/`, 2024.

[23] LangChain Team. Langchain documentation. `https://python.langchain.com/`, 2024.

[24] LangChain. Handoffs. `https://docs.langchain.com/oss/python/langchain/multi-agent/handoffs`. Accessed September 15, 2026.

[25] LangChain. Interrupts: human-in-the-loop workflows. `https://docs.langchain.com/oss/python/langgraph/interrupts`. Accessed September 15, 2026.

[26] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[27] Farida Siddiqi Prity, MD. Mehadi Hasan, Shakhawat Hossain Saif, Md. Maruf Hossain, Sazzad Hossain Bhuiyan, Md. Ariful Islam, and Md Tousif Hasan Lavlu. Enhancing agricultural productivity: A machine learning approach to crop recommendations. *Human-Centric Intelligent Systems*, 4:497–510, 2024.

[28] Charlotte Pelletier, Geoffrey I Webb, and François Petitjean. Temporal convolutional neural network for the classification of satellite image time series. *Remote Sensing*, 11(5):523, 2019.

[29] Marc Rußwurm, Charlotte Pelletier, Maximilian Zollner, Sébastien Lefèvre, and Marco Körner. Breizhcrops: A time series dataset for crop type mapping. In *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, volume V-2-2020, pages 1545–1551, 2020.

[30] Transfer learning, 2020. General reference on transfer learning / fine-tuning of pretrained models — replace with your preferred survey.

[31] European Space Agency. Sentinel-2 mission. `https://sentinel.esa.int/web/sentinel/missions/sentinel-2`, 2024.

[32] John W Rouse, Robert H Haas, John A Schell, and Donald W Deering. Monitoring vegetation systems in the great plains with erts. *NASA Special Publication*, 351:309, 1974.

[33] Stuart K McFeeters. The use of the normalized difference water index (ndwi) in the delineation of open water features. *International journal of remote sensing*, 17(7):1425–1432, 1996.

[34] Bo-Cai Gao. Ndwi—a normalized difference water index for remote sensing of vegetation liquid water from space. *Remote sensing of environment*, 58(3):257–266, 1996.

[35] Etalab. Données cadastrales ouvertes. `https://cadastre.data.gouv.fr/`. Accessed September 15, 2026.

[36] IGN. Registre Parcellaire Graphique (RPG). `https://geoservices.ign.fr/rpg`. Accessed September 15, 2026.

[37] IGN. LiDAR HD. `https://geoservices.ign.fr/lidarhd`. Accessed September 15, 2026.

[38] IGN. BD ORTHO. `https://geoservices.ign.fr/bdortho`. Accessed September 15, 2026.

[39] Open-Meteo. Weather Forecast API documentation. `https://open-meteo.com/en/docs`. Accessed September 15, 2026.

[40] ISRIC – World Soil Information. SoilGrids: Global gridded soil information. `https://isric.org/explore/soilgrids`. Accessed September 15, 2026.

[41] Food and Agriculture Organization of the United Nations. FAOSTAT. `https://www.fao.org/faostat/en/`. Accessed September 15, 2026.

[42] Tavily. Tavily documentation. `https://docs.tavily.com/documentation/about`. Accessed September 15, 2026.

[43] COMIFER. Méthode du bilan prévisionnel de l'azote (comifer). `https://www.comifer.asso.fr/`, 2023.

[44] Richard G Allen, Luis S Pereira, Dirk Raes, and Martin Smith. Crop evapotranspiration - guidelines for computing crop water requirements. Technical report, FAO Irrigation and drainage paper 56, 1998.

[45] Multi-criteria decision analysis, 2002. General reference on MCDA methods — replace with your preferred survey (e.g., Belton & Stewart, 2002).

[46] European Parliament and Council of the European Union. Regulation (EU) 2024/1689: Artificial Intelligence Act. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng`. Accessed September 15, 2026.

[47] Ragas developers. Ragas documentation. `https://docs.ragas.io/en/stable/`. Accessed September 15, 2026.

[48] scikit-learn developers. scikit-learn: Machine learning in Python. `https://scikit-learn.org/stable/`. Accessed September 15, 2026.

[49] SciPy developers. SciPy documentation. `https://docs.scipy.org/doc/scipy/`. Accessed September 15, 2026.

[50] pytest developers. pytest documentation. `https://docs.pytest.org/en/stable/`. Accessed September 15, 2026.

[51] Sebastián Ramírez. Fastapi documentation. `https://fastapi.tiangolo.com/`, 2024.

[52] Inc. Meta Platforms. React. `https://reactjs.org`, 2025. Accessed April 2025.

[53] Vite Team. Vite documentation. `https://vitejs.dev/`, 2024.

[54] TanStack Team. Tanstack documentation. `https://tanstack.com/`, 2024.

[55] Docker, Inc. Docker compose documentation. `https://docs.docker.com/compose/`, 2024.

[56] Linus Torvalds and the Git community. Git. `https://git-scm.com`, 2025. Accessed April 2025.